

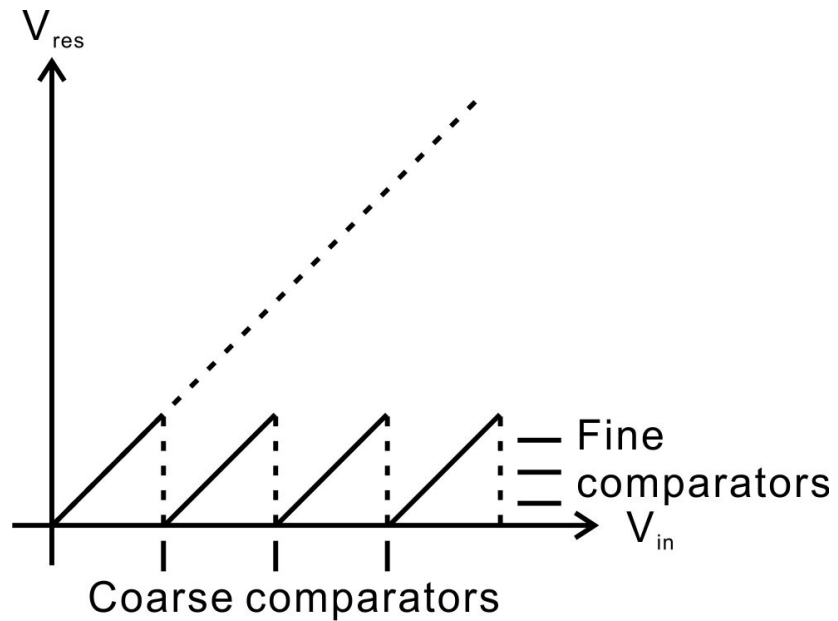
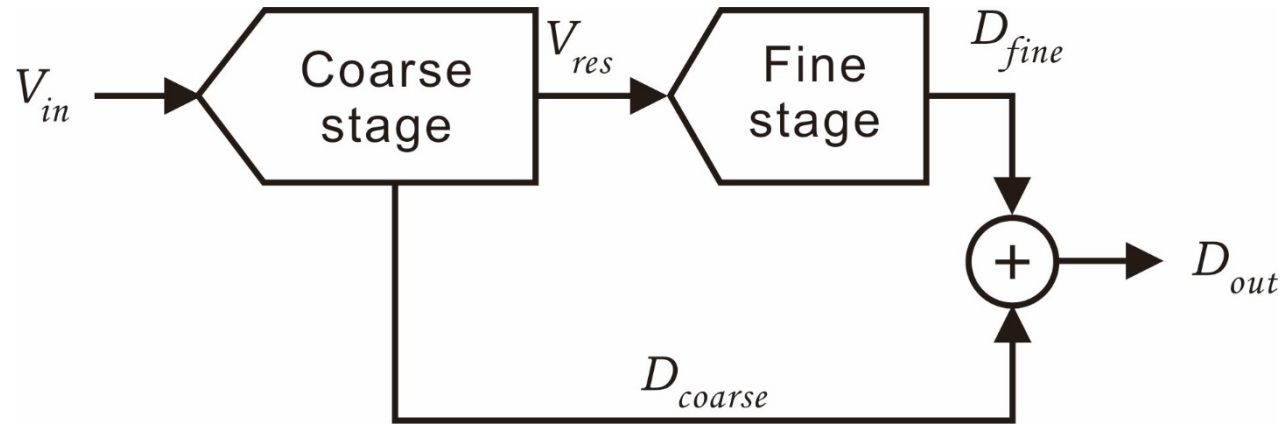
Pipelined ADC Concept

Nan Sun
Tsinghua University
nansun@tsinghua.edu.cn

Objective

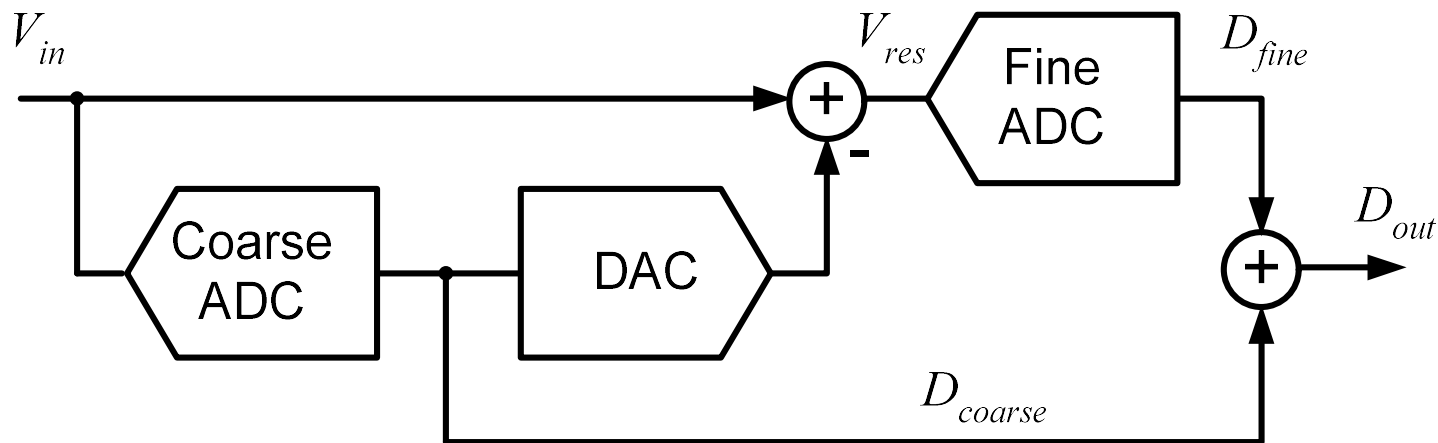
- Want an ADC with
 - High speed
 - High resolution
 - Low hardware complexity
 - Reduce number of comparators
 - Relax the requirement on comparator precision

Similar Idea as Folding ADC

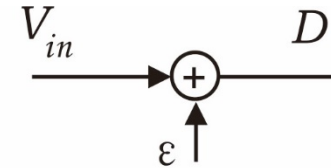
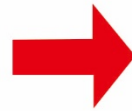
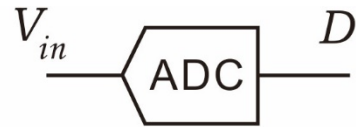


Two-Step Conversion

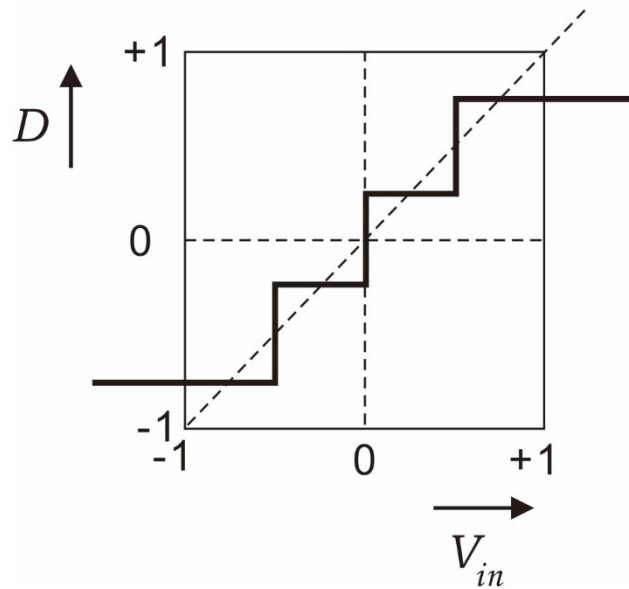
- General idea
 1. Perform a "coarse" quantization of the input
 2. Compute residual of step 1 conversion using a DAC and a subtractor
 3. Digitize residual using a "fine" quantizer and digitally add to output



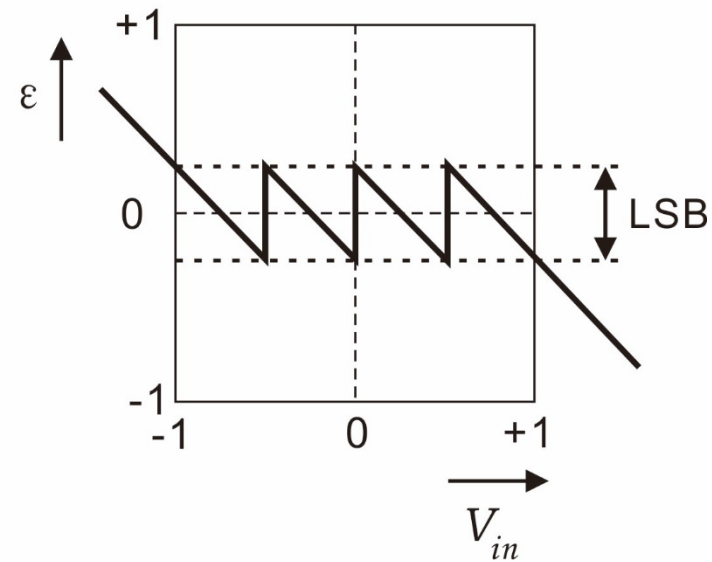
Quantizer Model



Transfer function

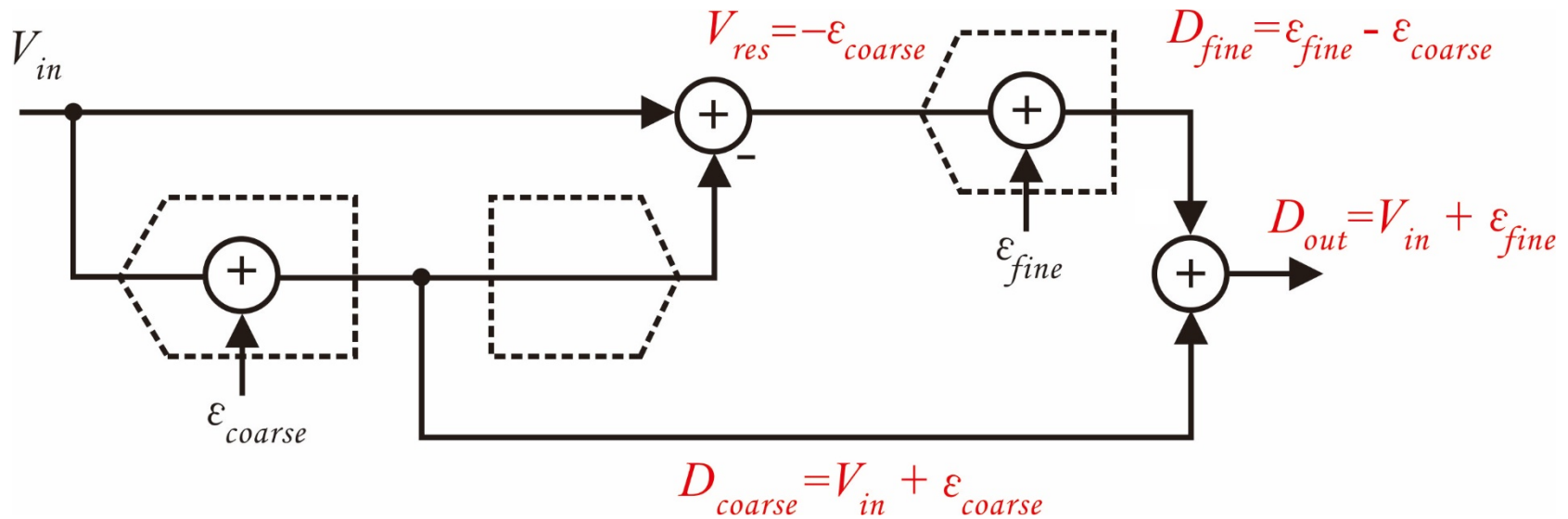


Quantization Error



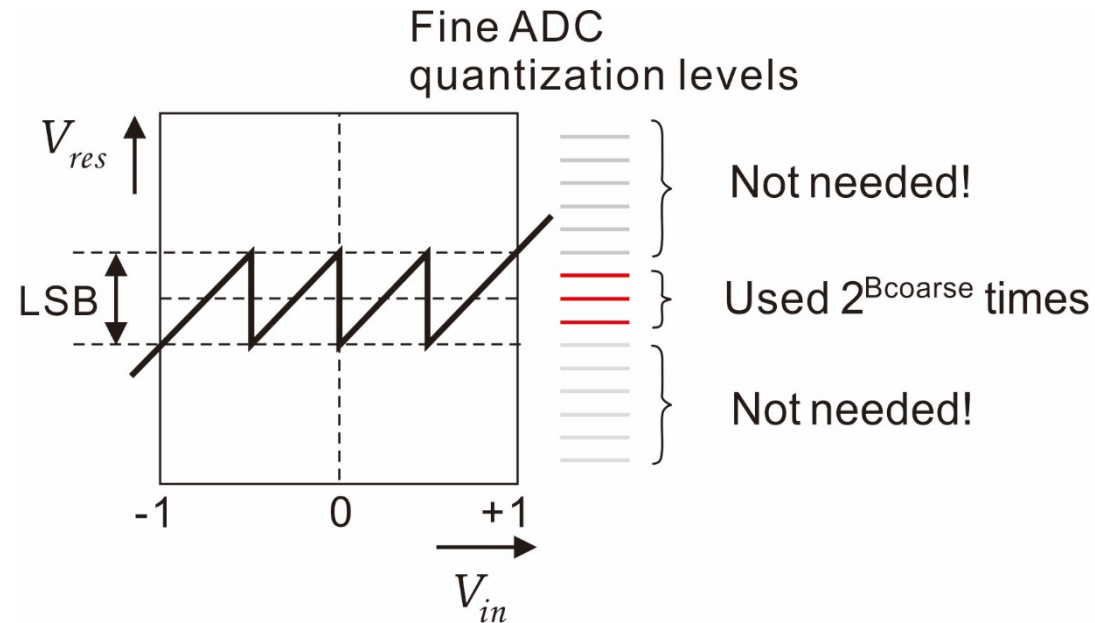
Analysis

- Assuming ideal DAC



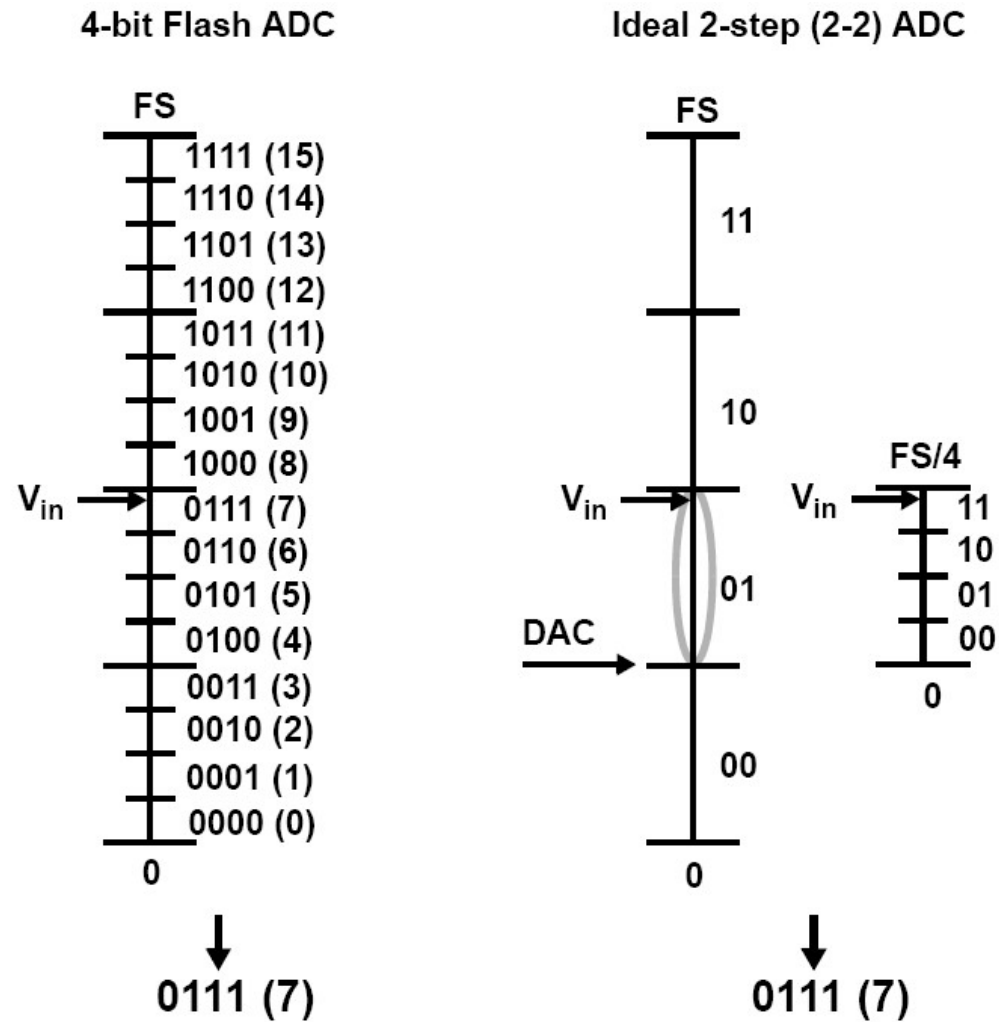
- Output contains only quantization error from fine ADC ☺

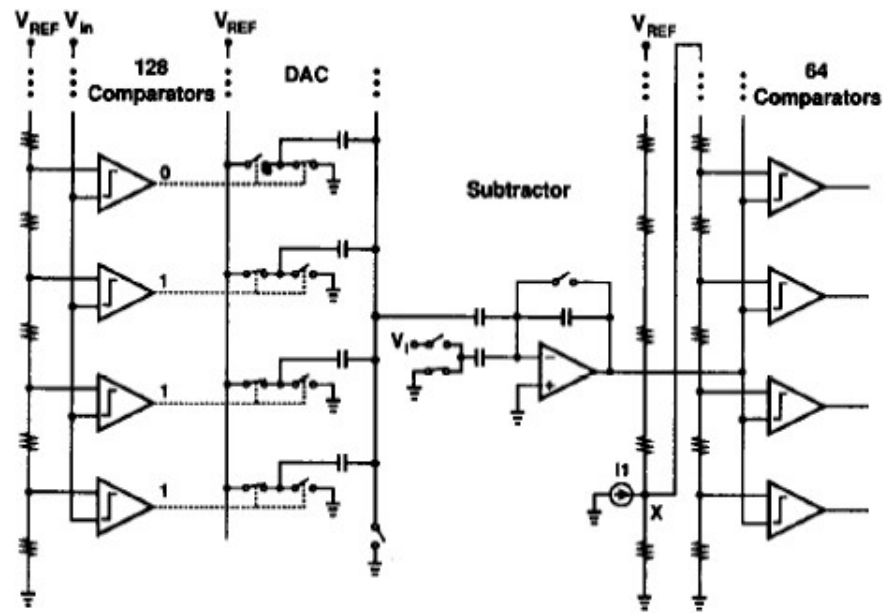
Input to Fine Quantizer (V_{res})



- Example
 - 2b coarse ADC and 2b fine ADC
 - Need only 6 comparators, compared to 15 in a 4-bit flash ADC
 - Advantage becomes more pronounced at higher resolutions

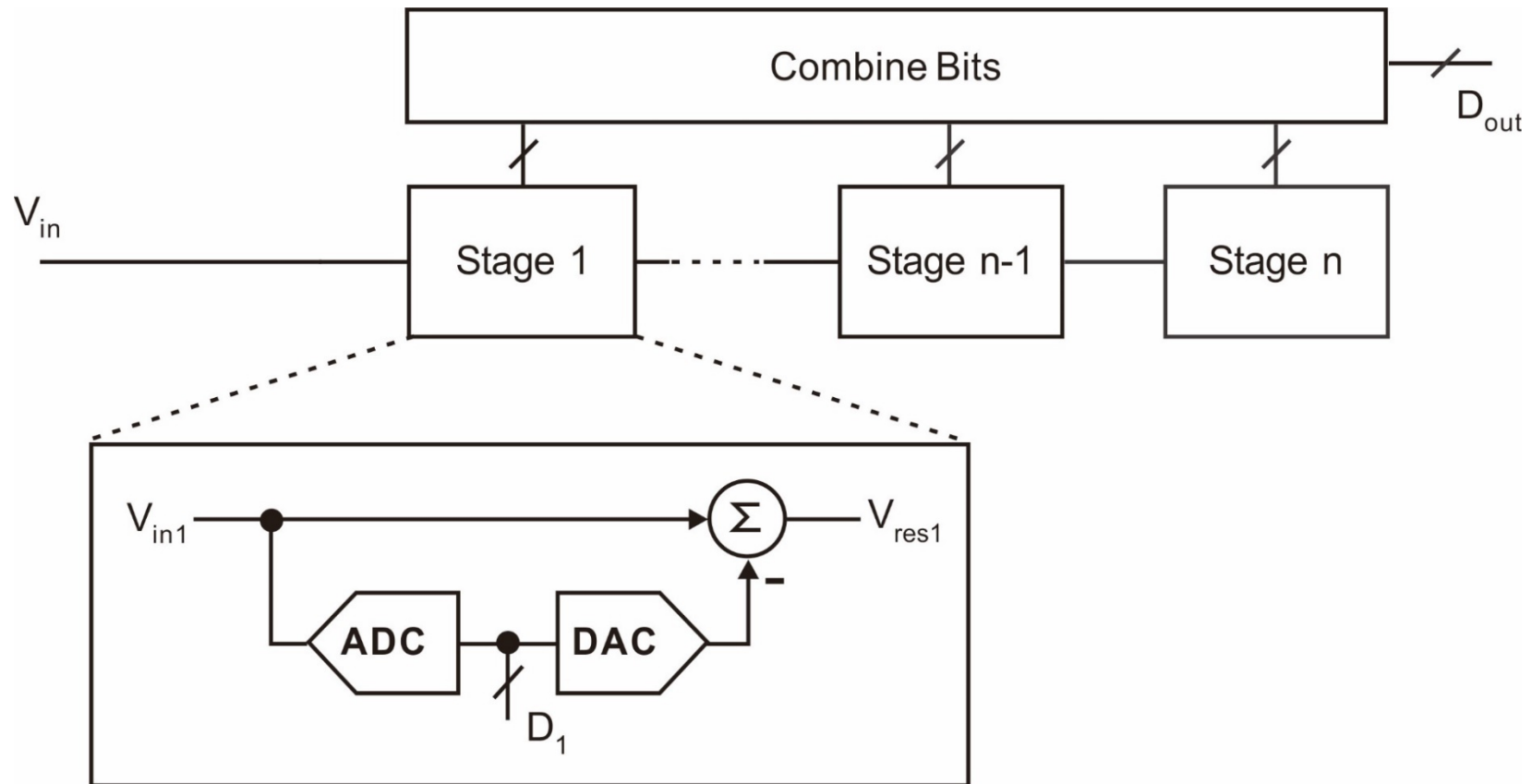
Alternative Illustration



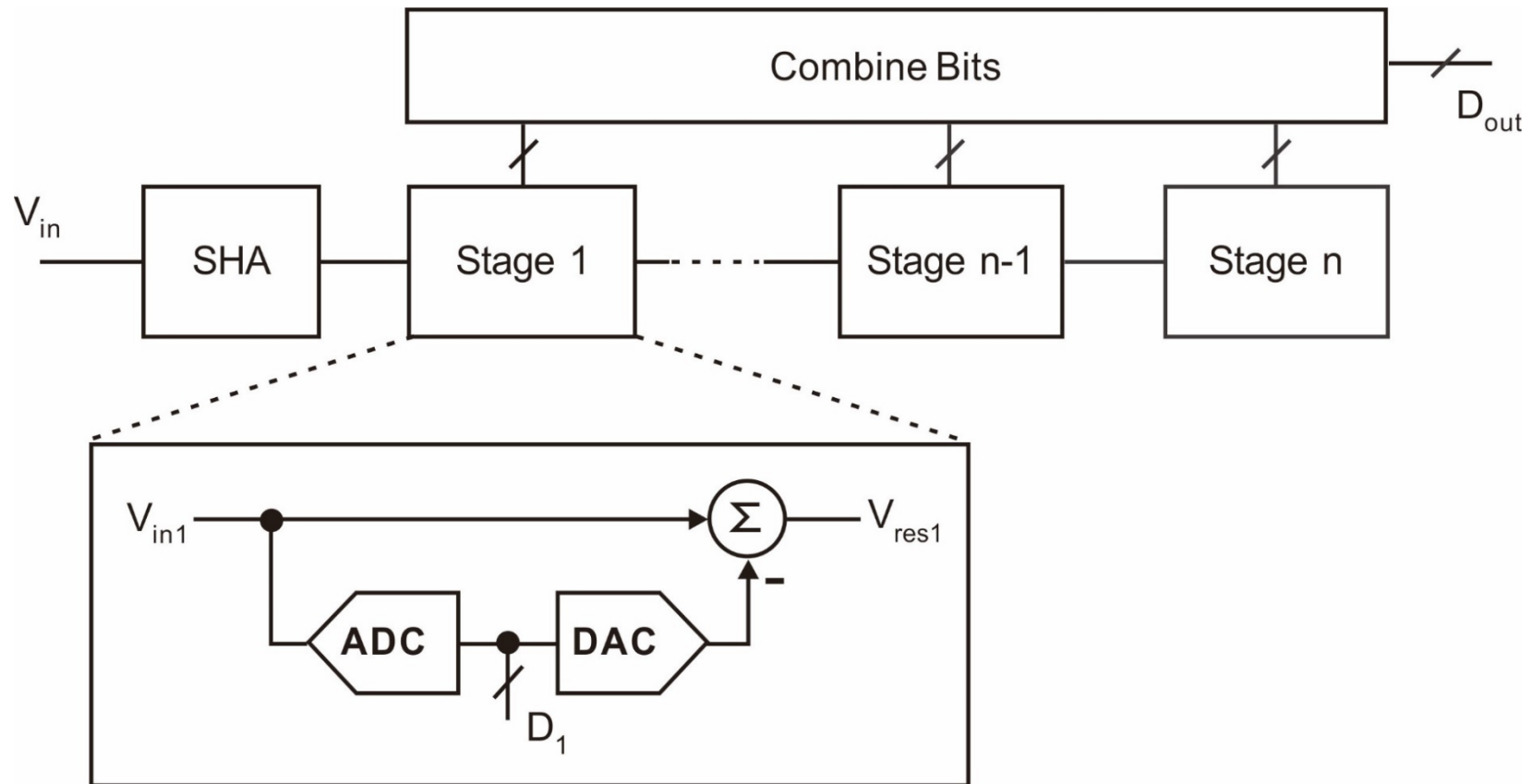


9

Multi-Step ADC

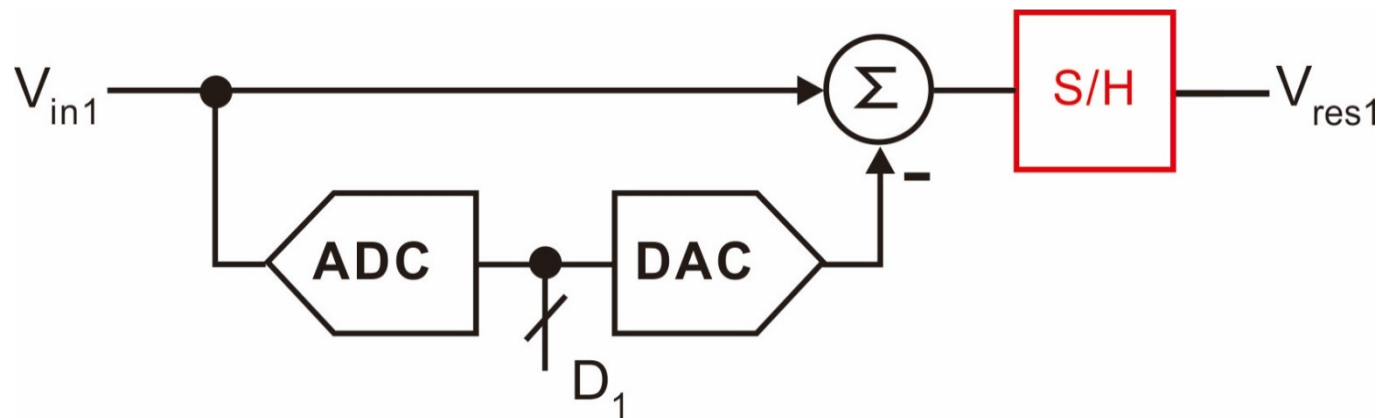


Multi-Step ADC



Increase Conversion Speed

- Long conversion time:
 - $T_{\text{conv}} \cong N \cdot (T_{\text{ADC}} + T_{\text{DAC}} + T_{\text{SUB}})$
- Solution
 - Turn delay into latency by pipelining
 - Add a S/H after each stage

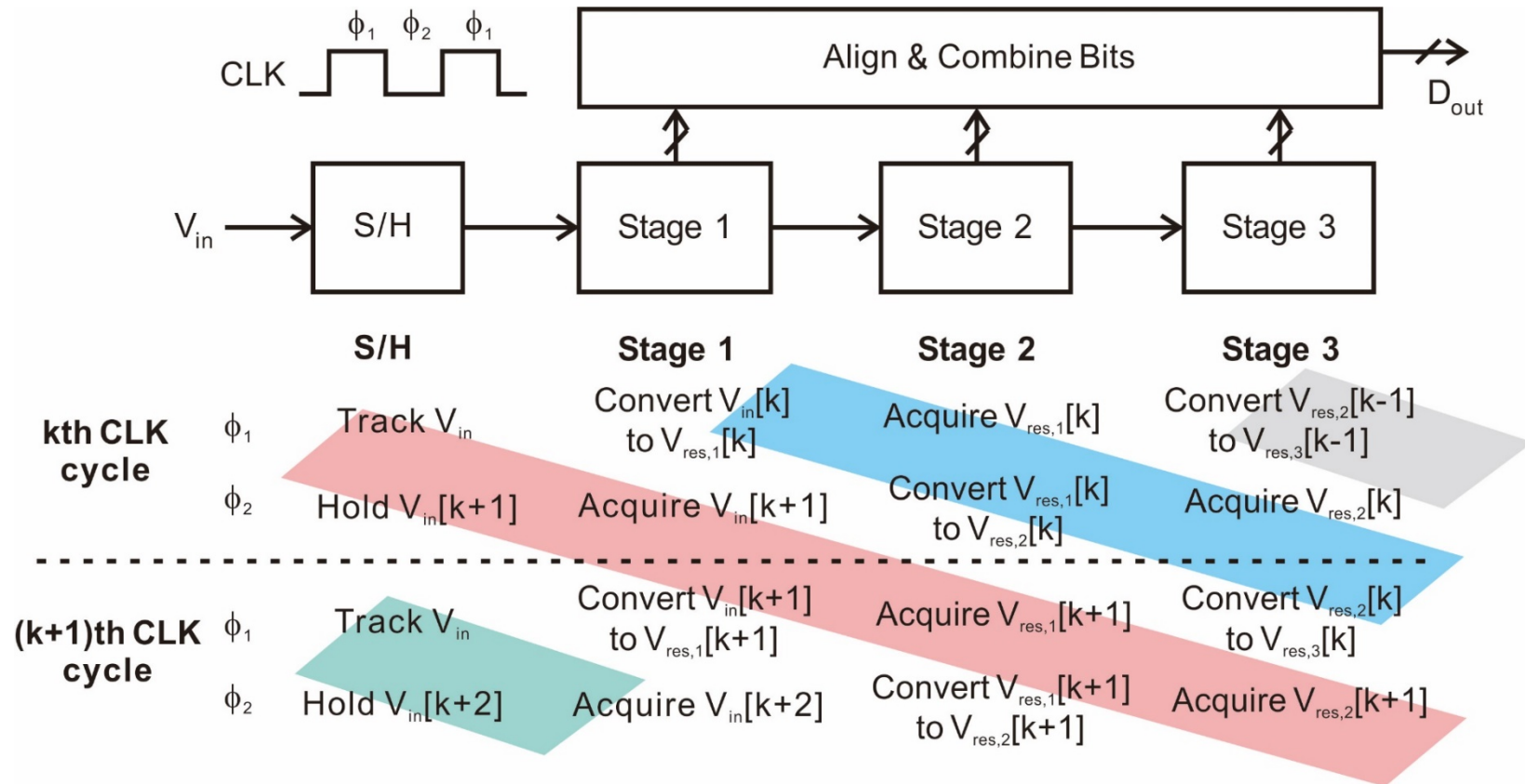


Pipelining – An Old Idea

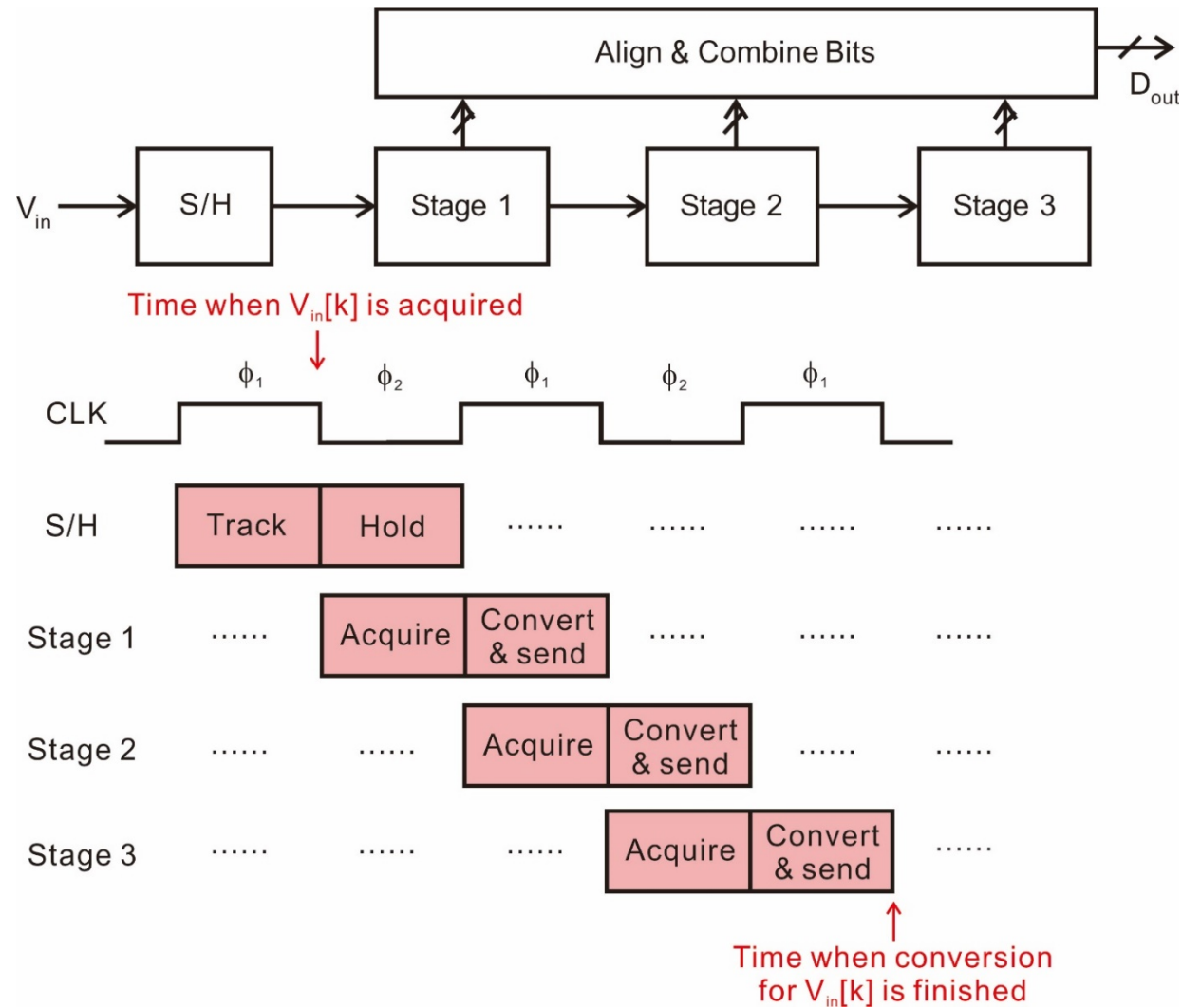


- At the Ford Motor Company's Highland Park, Mich., plant in 1913

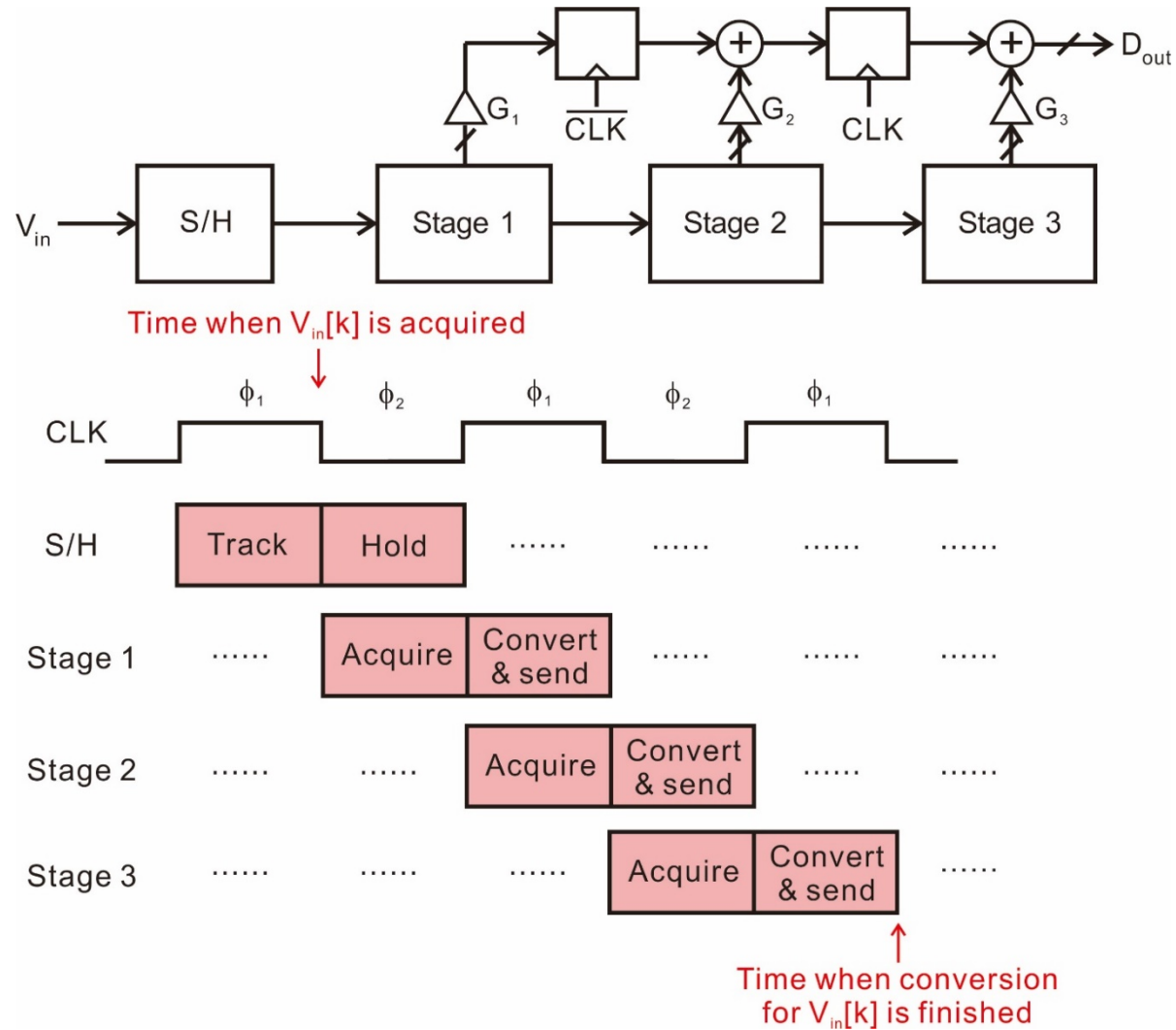
Pipelined Operation (1)



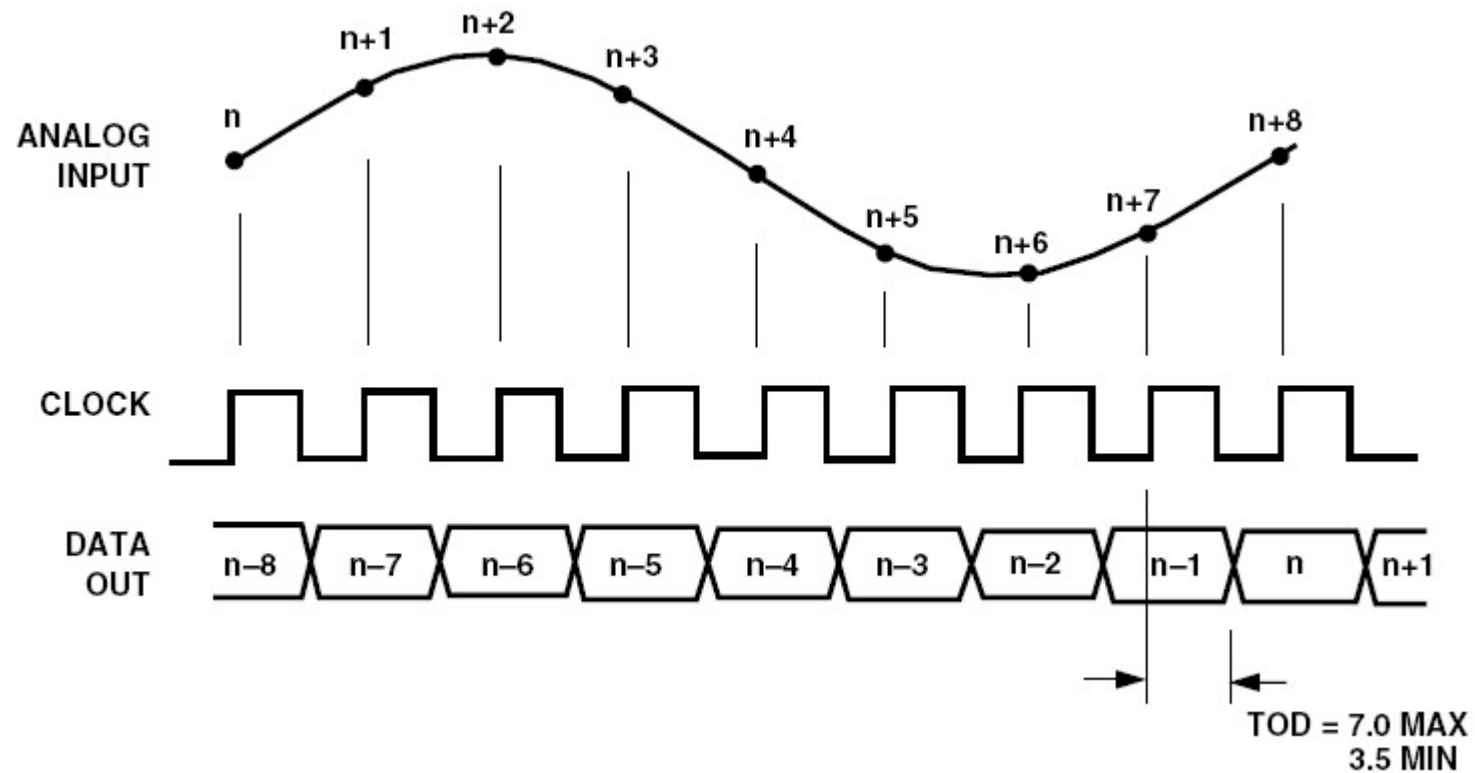
Pipelined Operation (2)



Bit Alignment



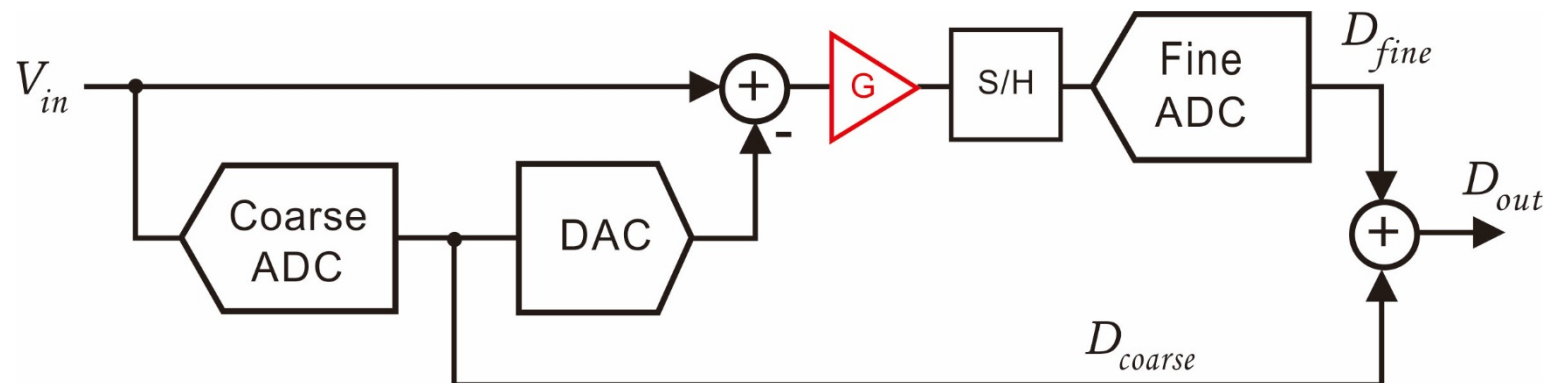
Latency



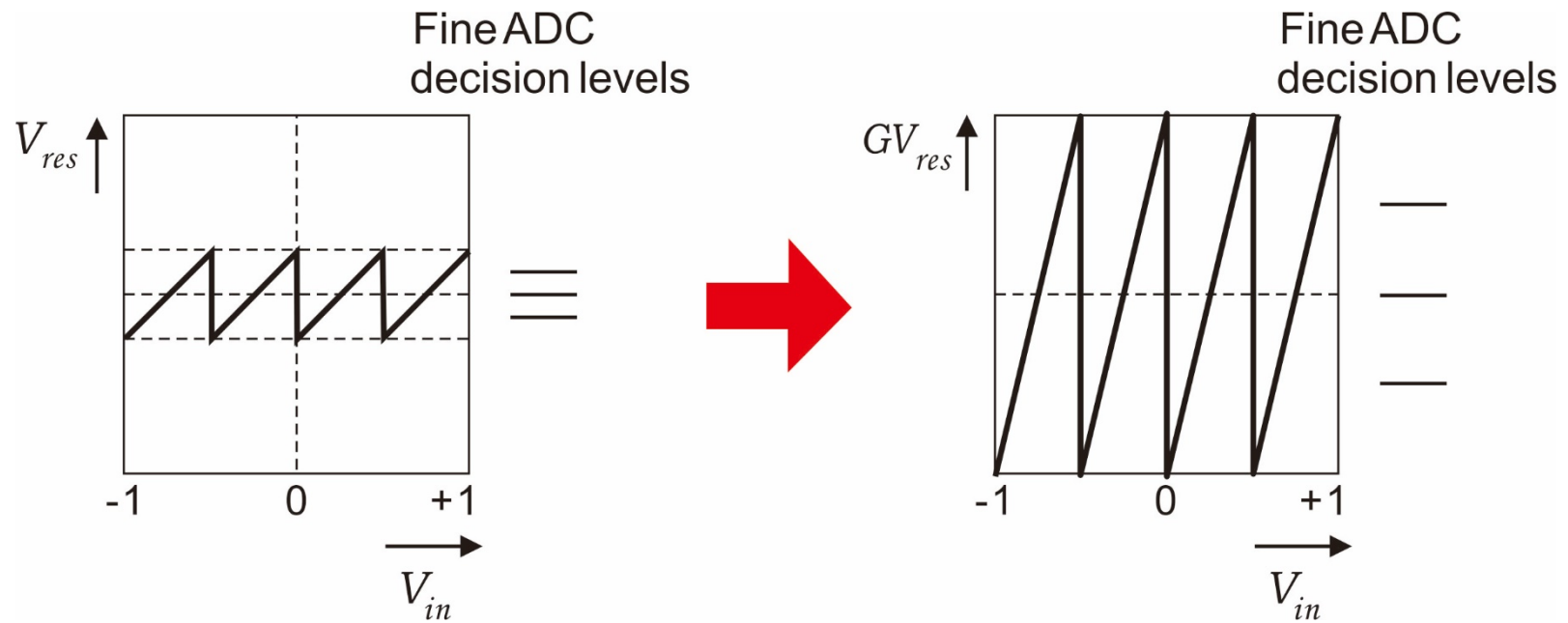
[Analog Devices, AD9226 Data Sheet]

Relax Comparator Requirement

- Fine ADC(s) must have high precision
 - E.g. 8-bit converter with 4-bit/4-bit partition; fine 4-bit decision levels must have "8-bit precision"
- Solution
 - Introduce gain after subtraction

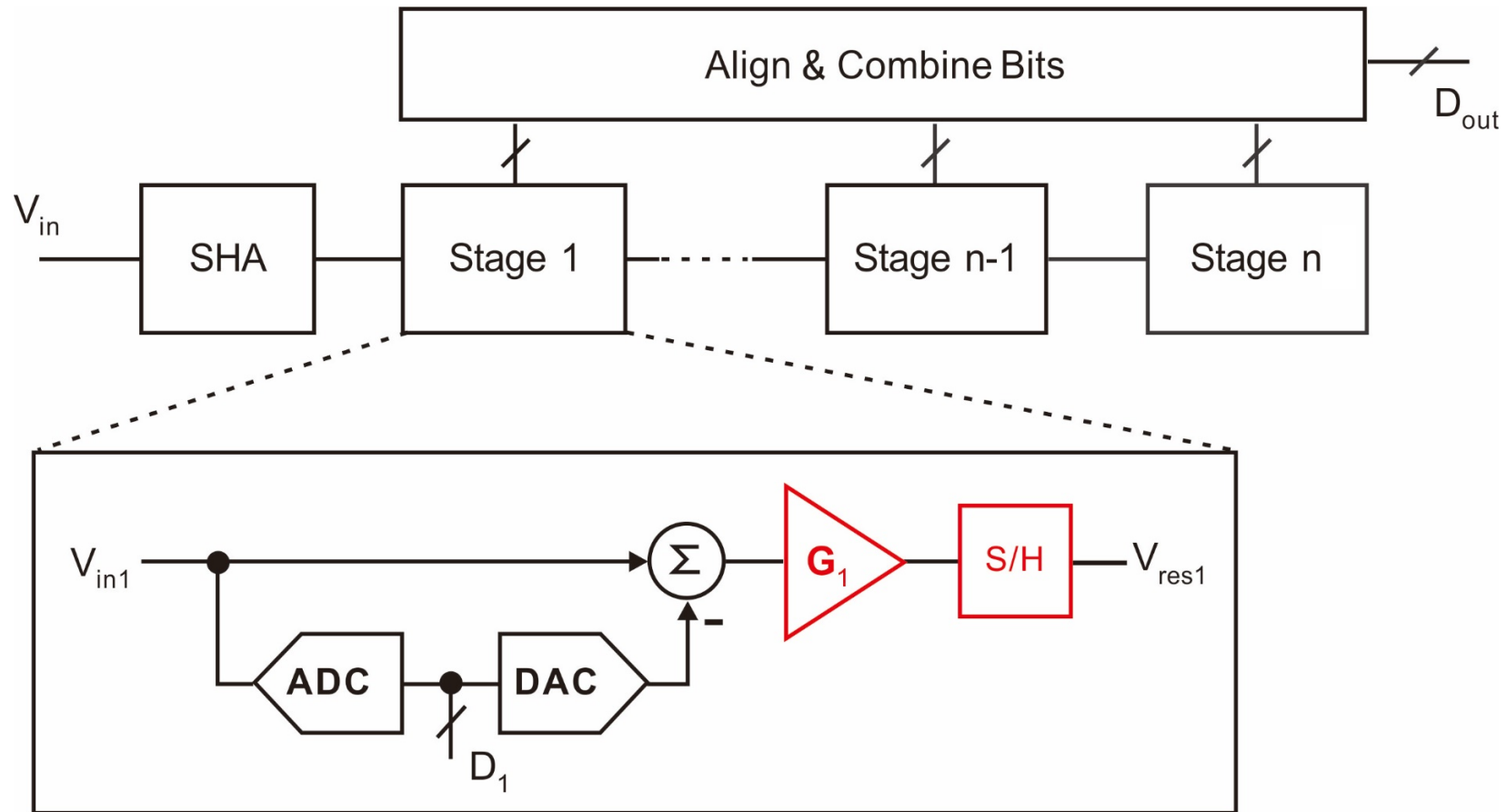


Input to Fine Quantizer with Gain



- No need for precision comparators

Pipeline ADC Block Diagram

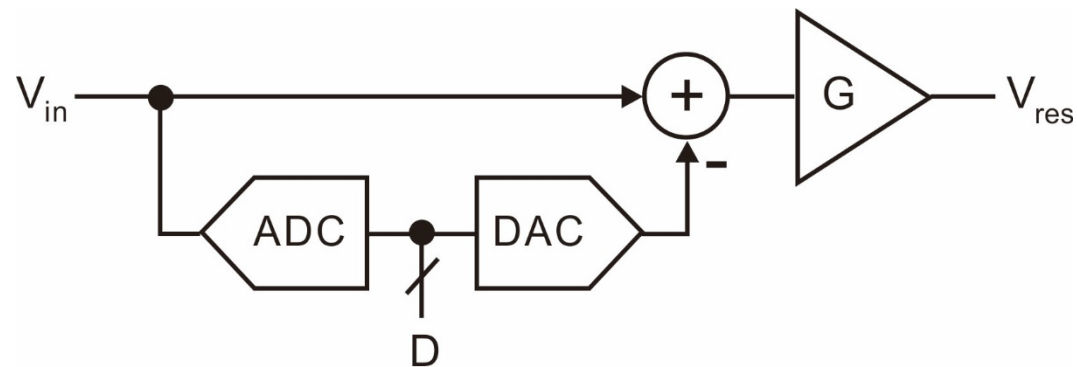


Pipeline ADC Characteristics

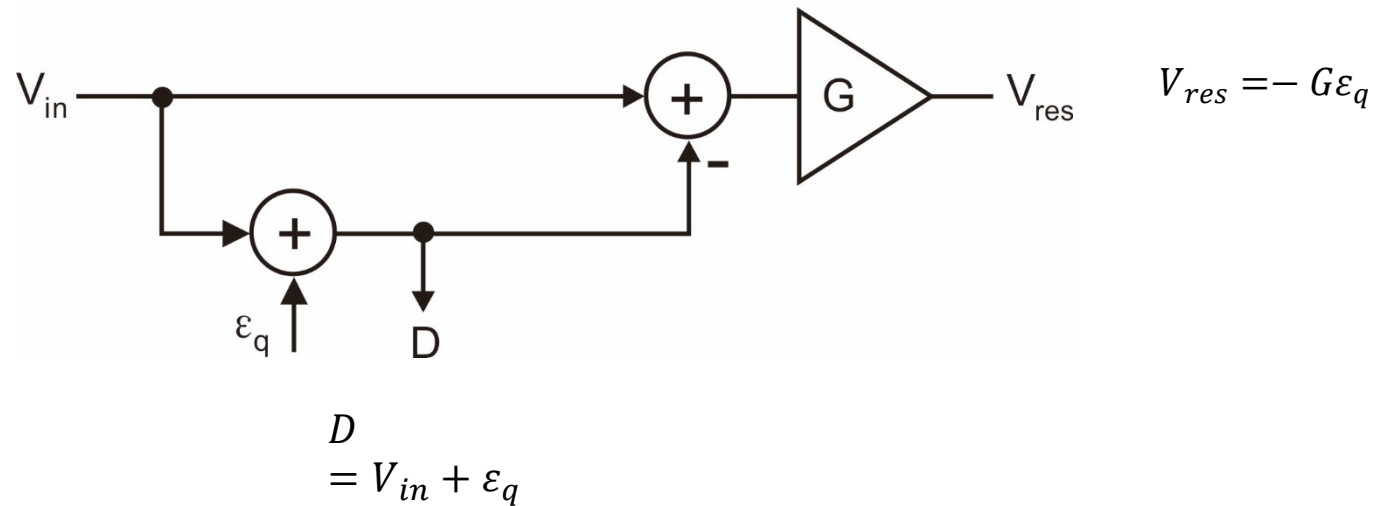
- Number of pipelined stages increases linearly with resolution
 - Hardware complexity $\sim B$
 - Flash ADC hardware complexity $\sim 2^B$
- Pipeline ADC trades latency for conversion speed
 - Conversion speed limited by only one stage delay
 - Enables high-speed operation
 - Latency can be an issue in some applications
 - E.g. in feedback control loops
- Pipelining only possible with good analog "memory elements"
 - Calls for CMOS implementation using switched-capacitor circuits

Stage Analysis

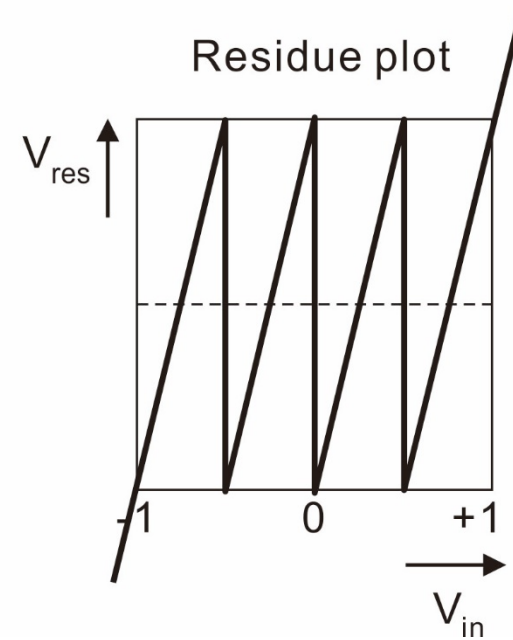
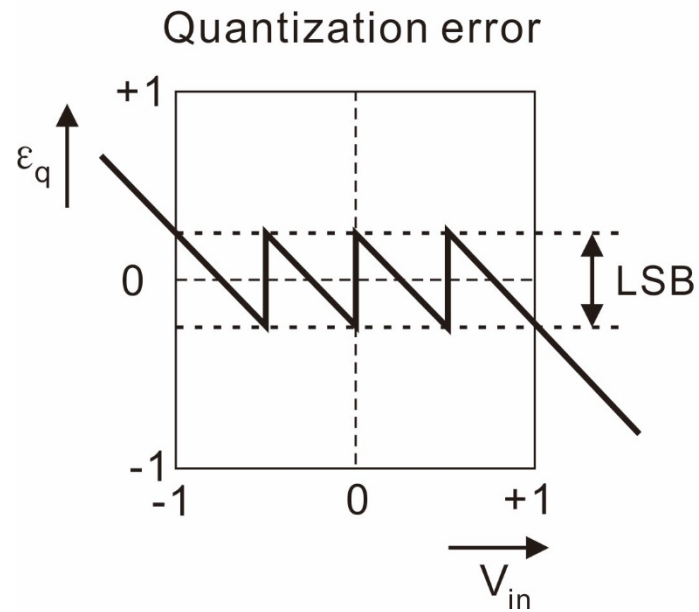
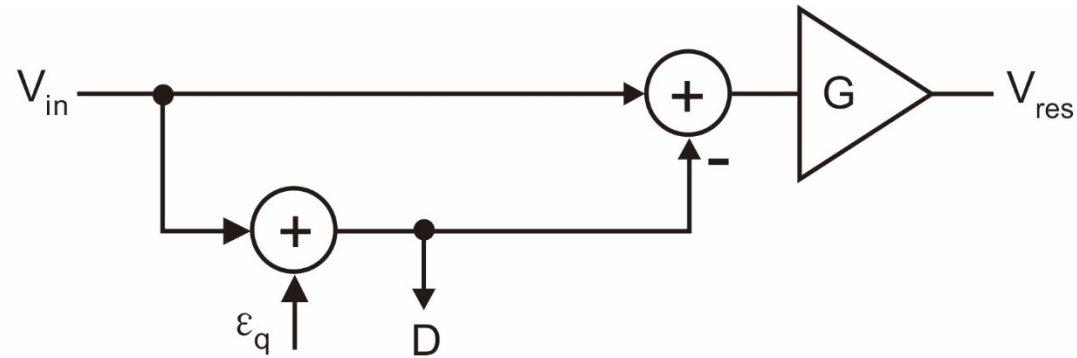
- Ignore timing/clock delays for simplicity



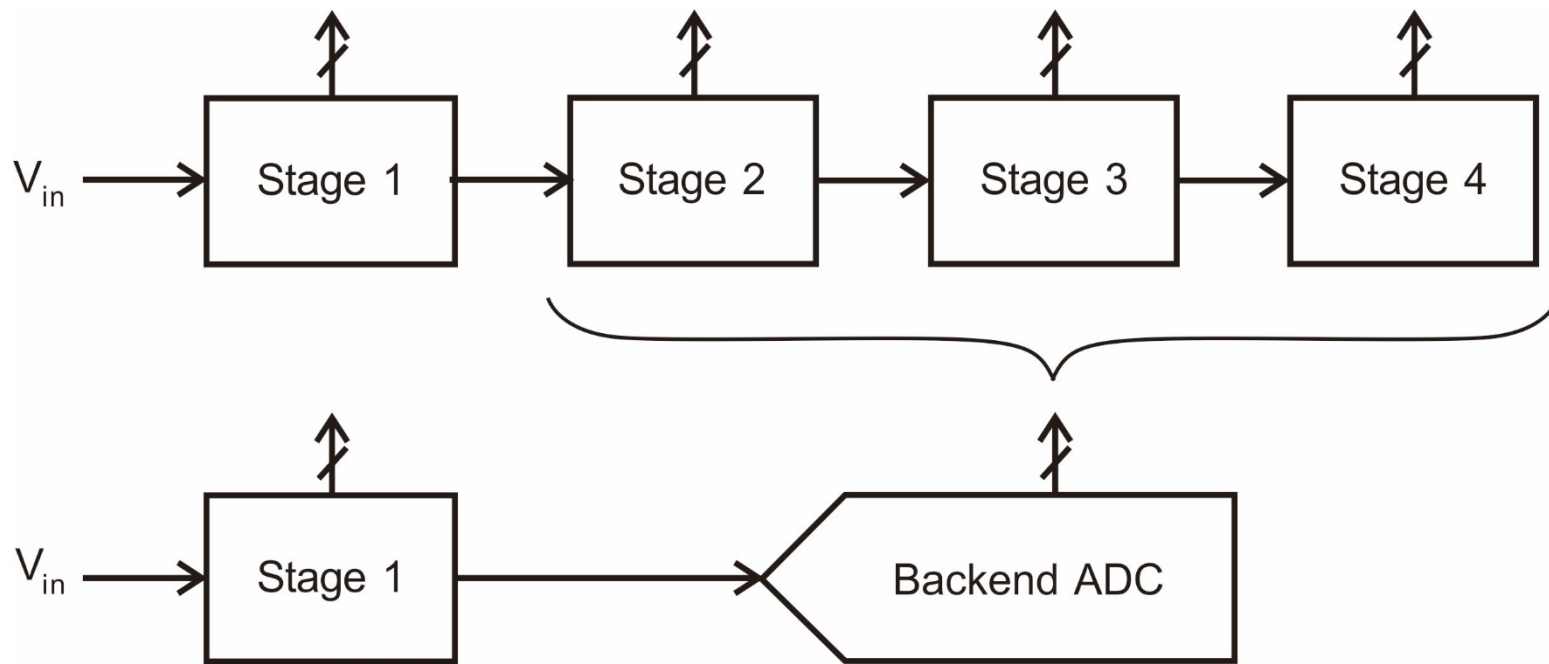
Model



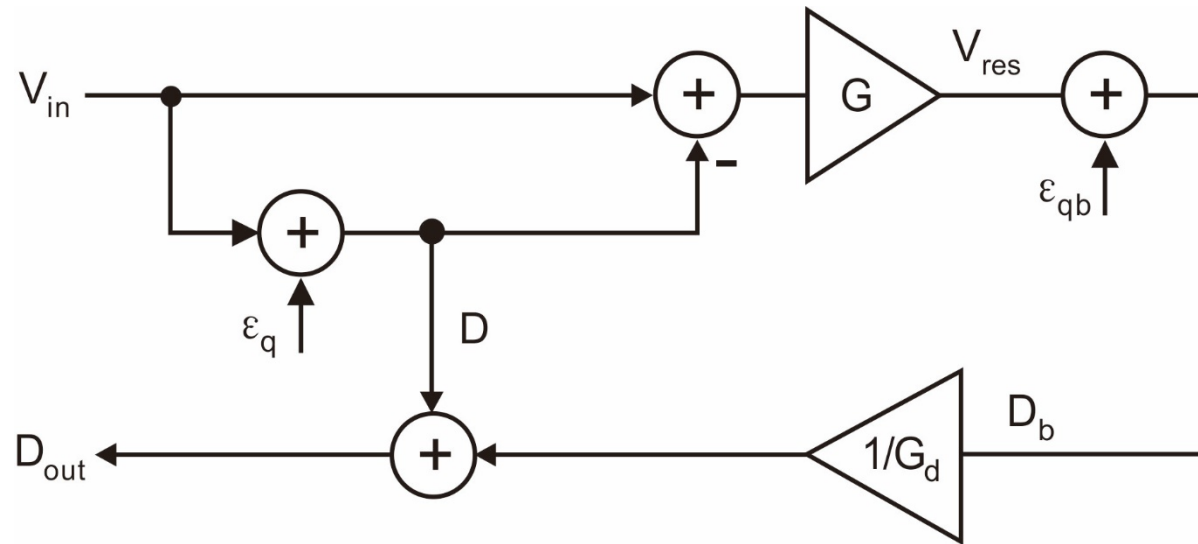
"Residue Plot" (2-bit Sub-ADC)



Pipeline Decomposition

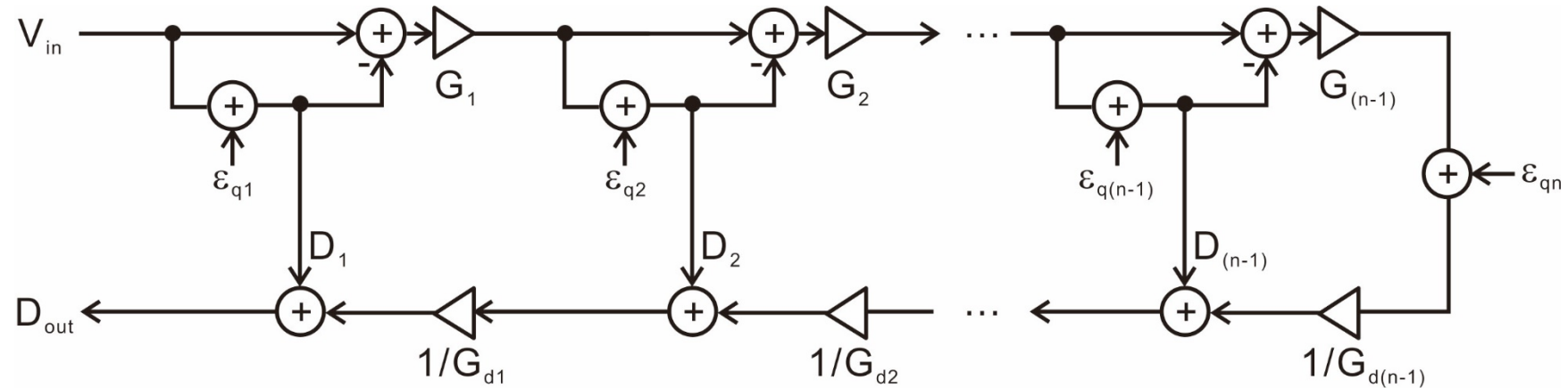


Resulting Model



$$\begin{aligned}
 D_{out} &= V_{in} + \varepsilon_q \left(1 - \frac{G}{G_d} \right) + \frac{\varepsilon_{qb}}{G_d} \\
 &= V_{in} + \frac{\varepsilon_{qb}}{G_d} \quad \text{for } G = G_d
 \end{aligned}$$

Extension



$$D_{out} = V_{in} + \epsilon_{q1} \left(1 - \frac{G_1}{G_{d1}} \right) + \frac{\epsilon_{q2}}{G_{d1}} \left(1 - \frac{G_2}{G_{d2}} \right) + \dots + \frac{\epsilon_{q(n-1)}}{\prod_{j=1}^{n-2} G_{dj}} \left(1 - \frac{G_{(n-1)}}{G_{d(n-1)}} \right) + \frac{\epsilon_{qn}}{\prod_{j=1}^{n-1} G_{dj}}$$

- With ideal DACs and matched analog/digital gains ($G_{dj}=G_j$)

$$D_{out} = V_{in} + \frac{\epsilon_{qn}}{\prod_{j=1}^{n-1} G_j} \Rightarrow B_{ADC} = B_n + \sum_{j=1}^{n-1} \log_2 G_j$$

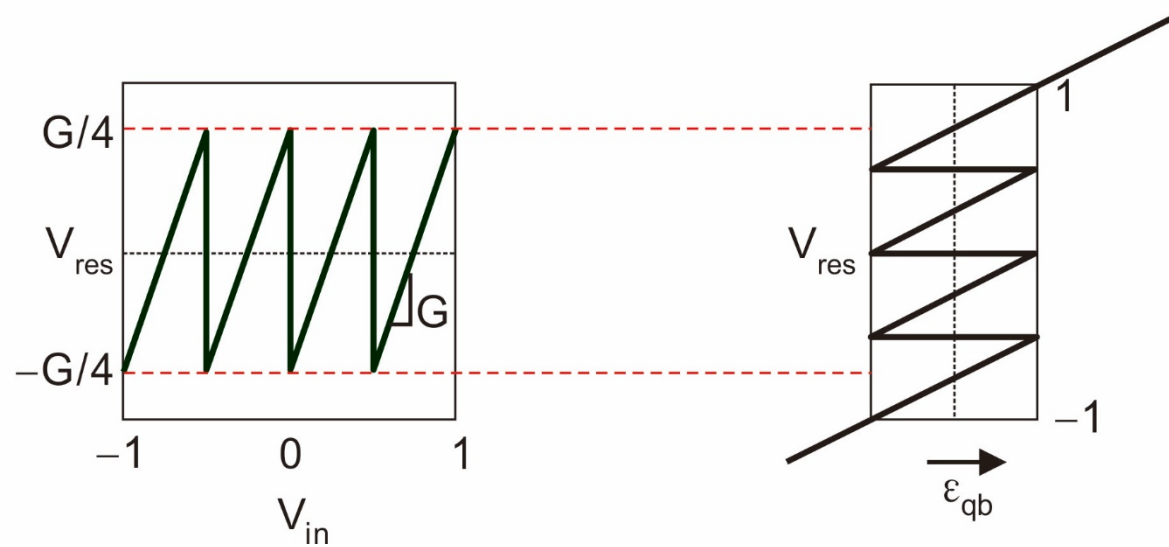
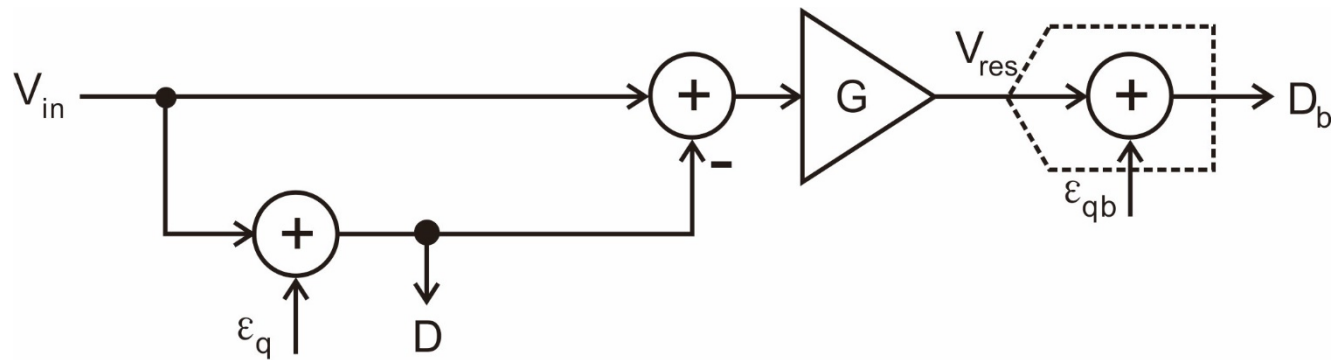
Analysis

- The only error in D_{out} is that of last quantizer, divided by aggregate gain
- Aggregate ADC resolution is independent of sub-ADC resolutions in stage 1 to stage (n-1).
- Gain is directly related to resolution
- Define "effective" resolution of the kth stage as $R_k = \log_2(G_k)$
 - $B_{\text{ADC}} = R_1 + R_2 + R_3 + \dots + R_{N-1} + B_N$

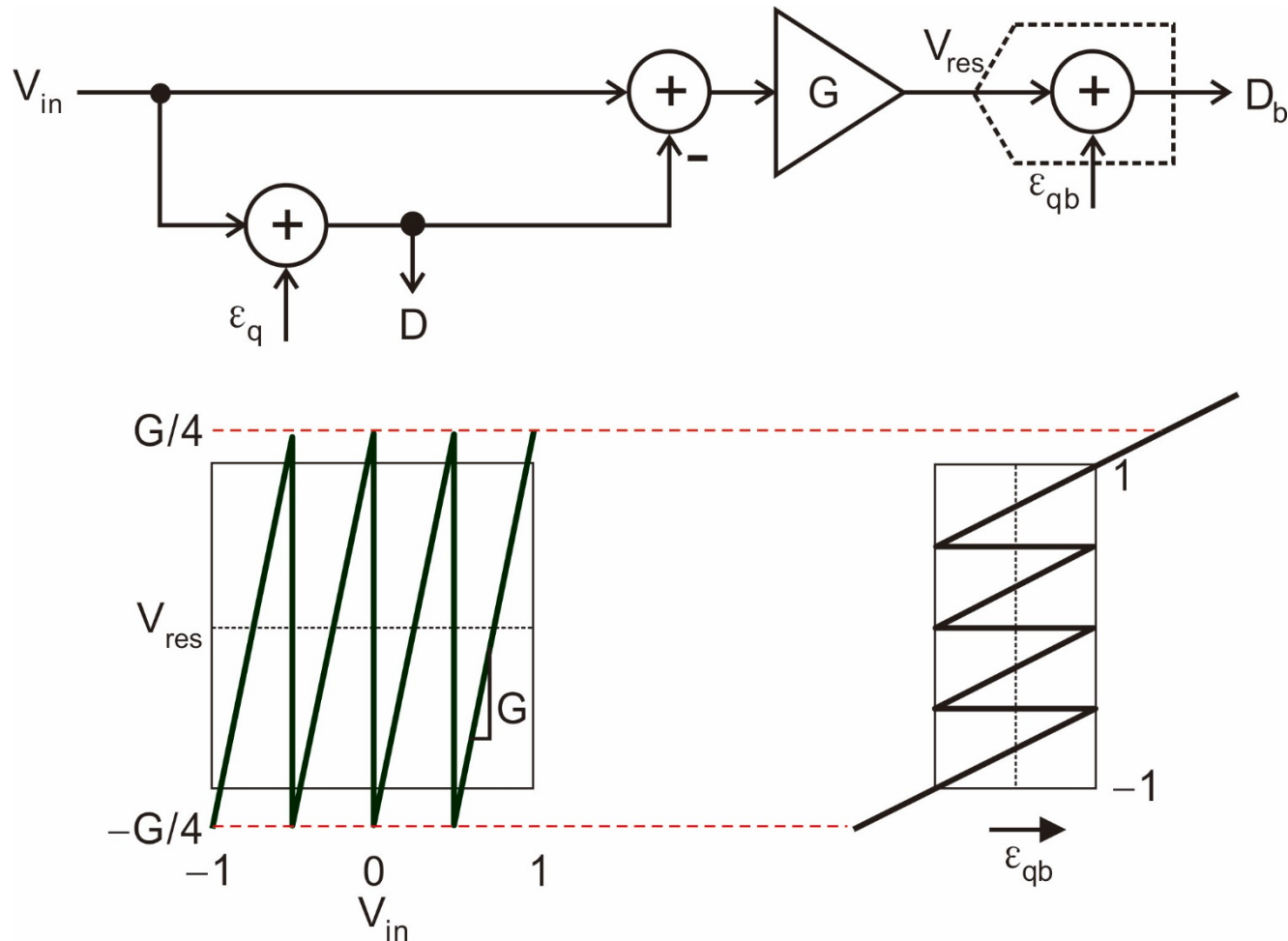
Questions

- How to pick inter-stage gain G for a given sub-ADC resolution?
- Nonidealities and solutions
 - Sub-ADC errors
 - Amplifier offset
 - Amplifier gain error
 - Sub-DAC error

Upper Bound for Stage Gain

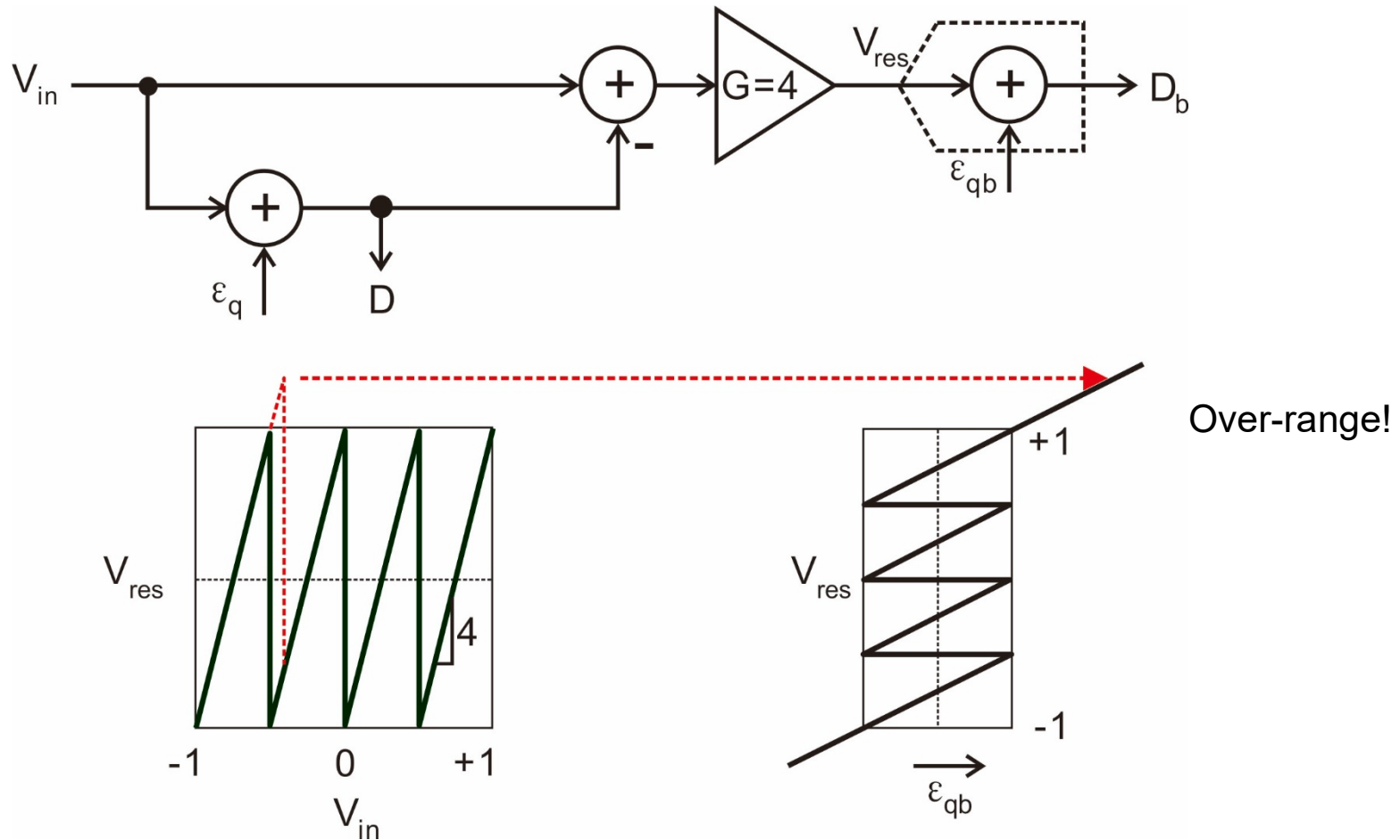


Upper Bound for Stage Gain



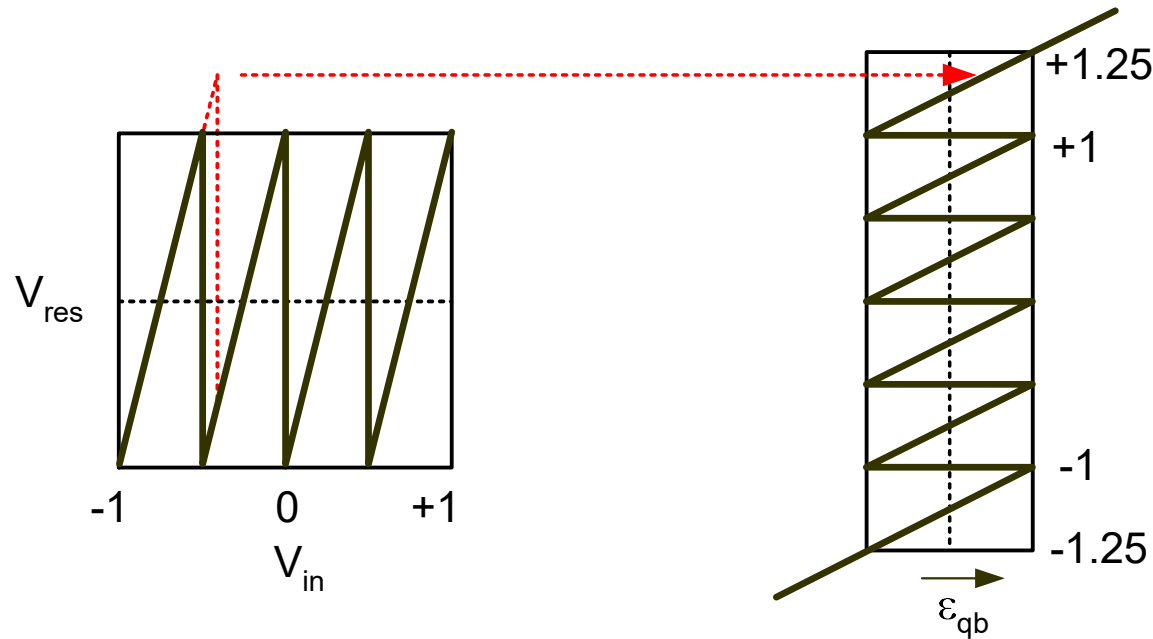
- Backend ADC will be overloaded for $G > 2^B$

Issue with $G=2^B$



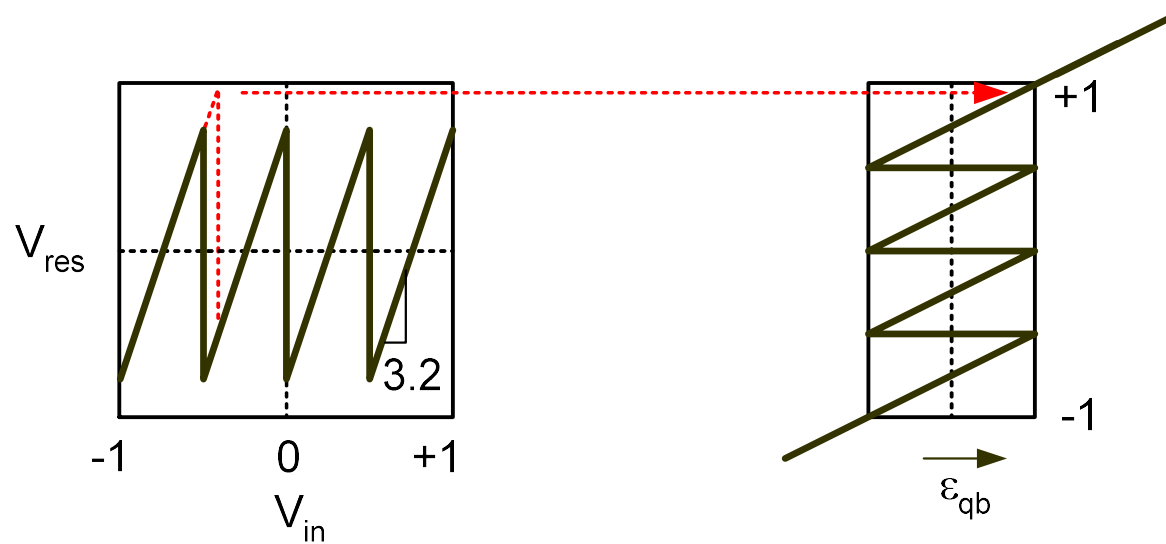
- Any error in sub-ADC decision levels (e.g. comparator offsets) will overload the backend ADC

Idea #1: $G=2^B$, extended backend



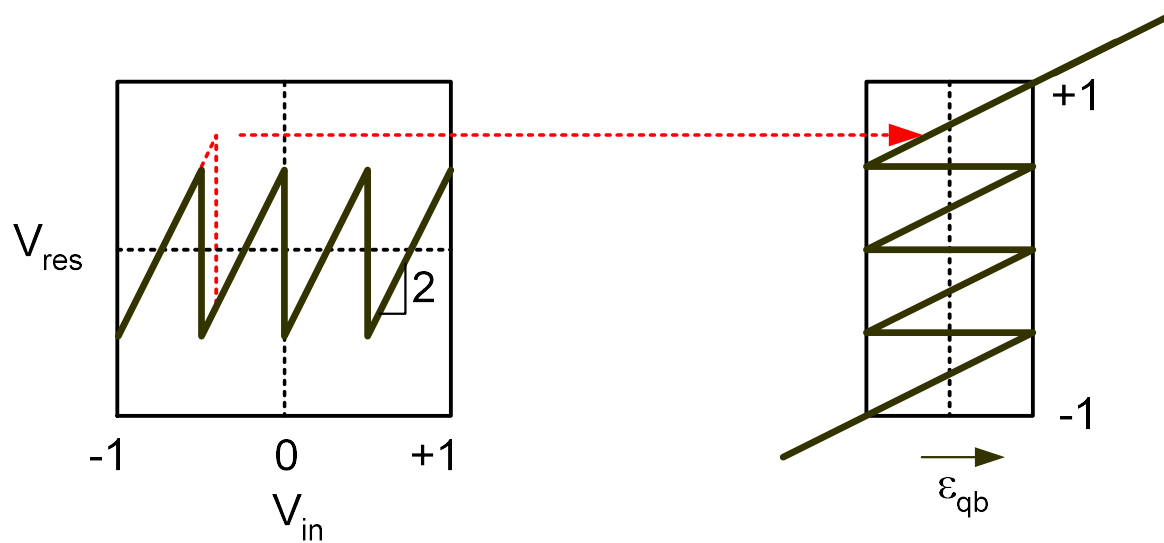
- Add extra decision levels in the backend stage
- See e.g. [Opris 1998]

Idea #2: G slightly less than 2^B



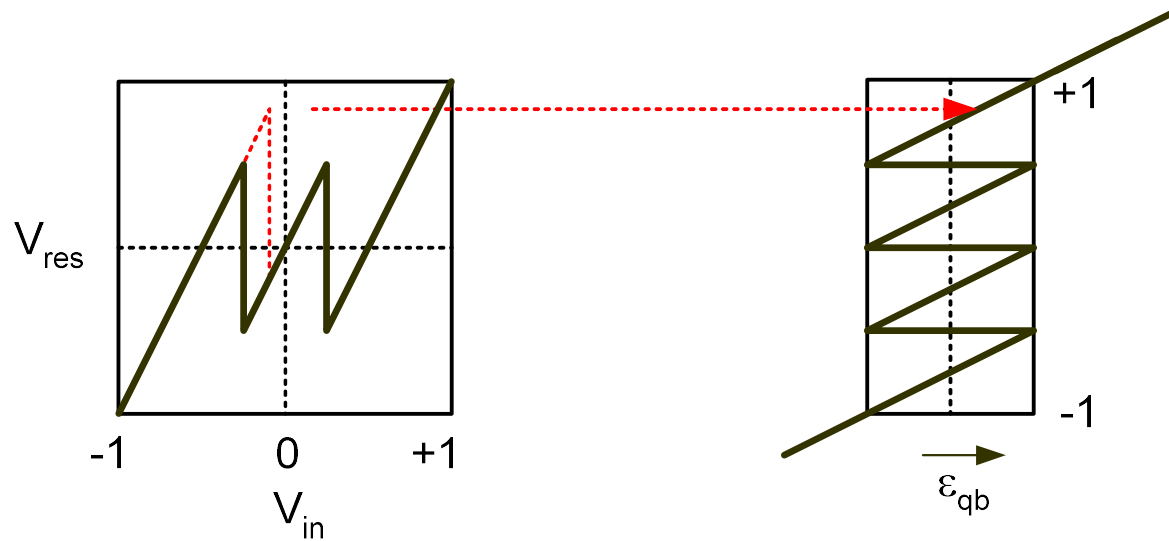
- Effective stage resolution can be non-integer ($R = \log_2 G$)
 - E.g. $R = \log_2 3.2 = 1.68$ bits
- See e.g. [Karanicolas 1993]

Idea #3: $G < 2^B$, but Power of Two



- Effective stage resolution is an integer
 - E.g. $R = \log_2 2 = 1 = B-1$
 - Digital hardware requires only a few adders, no need to implement fractional weights
- See e.g. [Mehr 2000]

Variant of Idea #3: "1.5-bit stage"

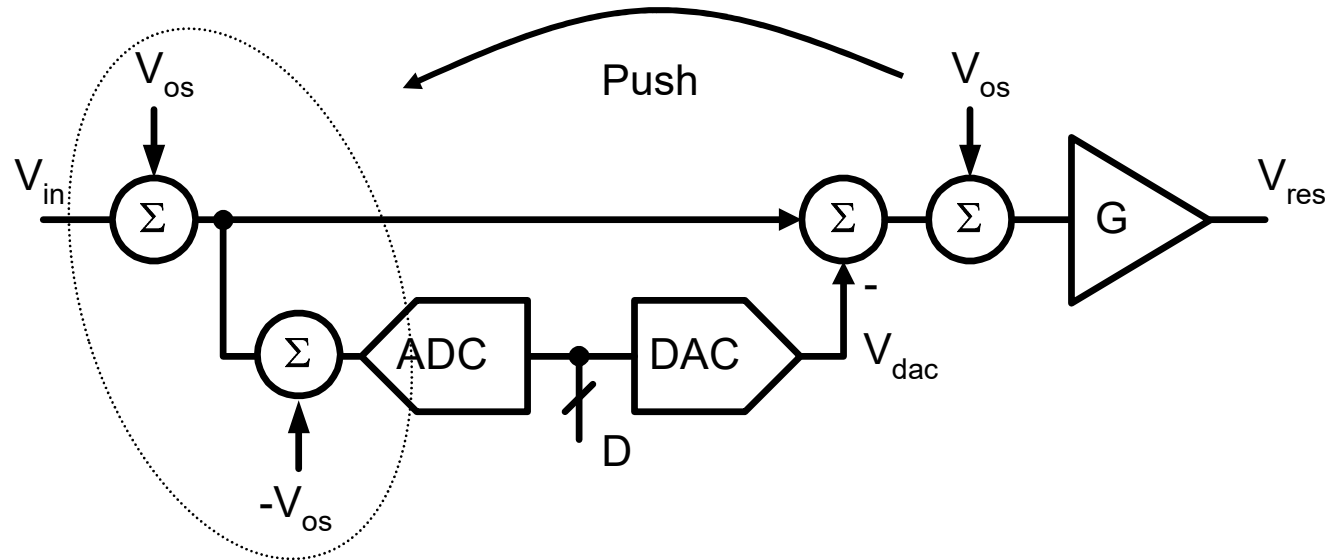


- Number of comparators reduced to 2
- DAC becomes easier to design
- $R = \log_2 2 = 1$ (effective stage resolution)
- See e.g. [Lewis 1992]

Summary on Sub-ADC Redundancy

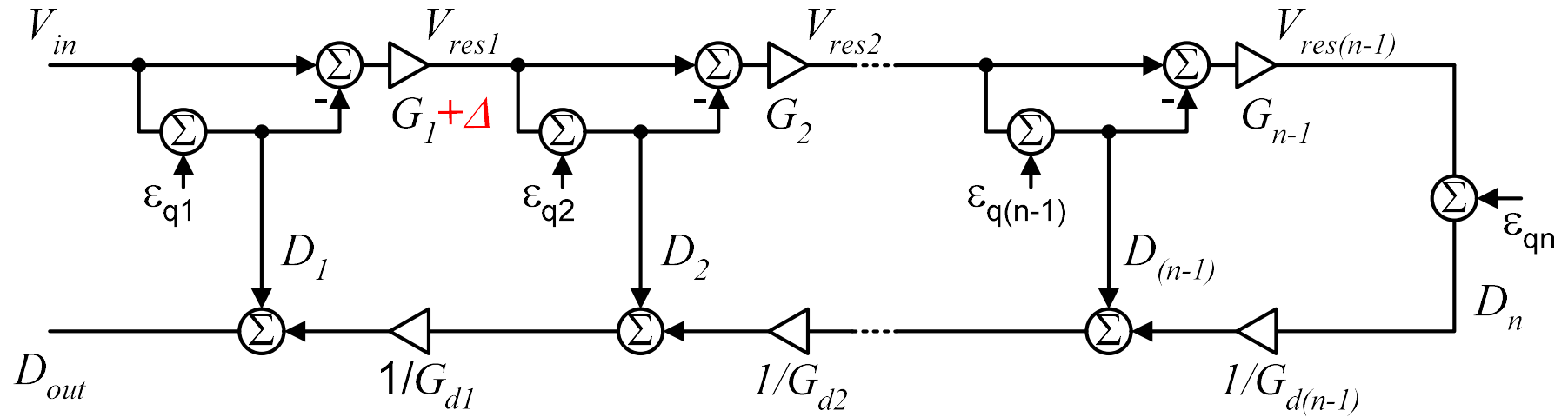
- We can tolerate sub-ADC errors as long as
 - The residue stays "inside the box", or
 - Another stage downstream returns the residue "into the box" before it reaches last quantizer
- This result applies to any stage in an n-stage pipelined ADC
 - Can always decompose pipeline into single stage + backend ADC

Amplifier Offset



- Amplifier offset can be referred toward stage input and results in
 - Global offset
 - Usually not a problem
 - Sub-ADC offset
 - Easily accommodated through redundancy

Gain Errors



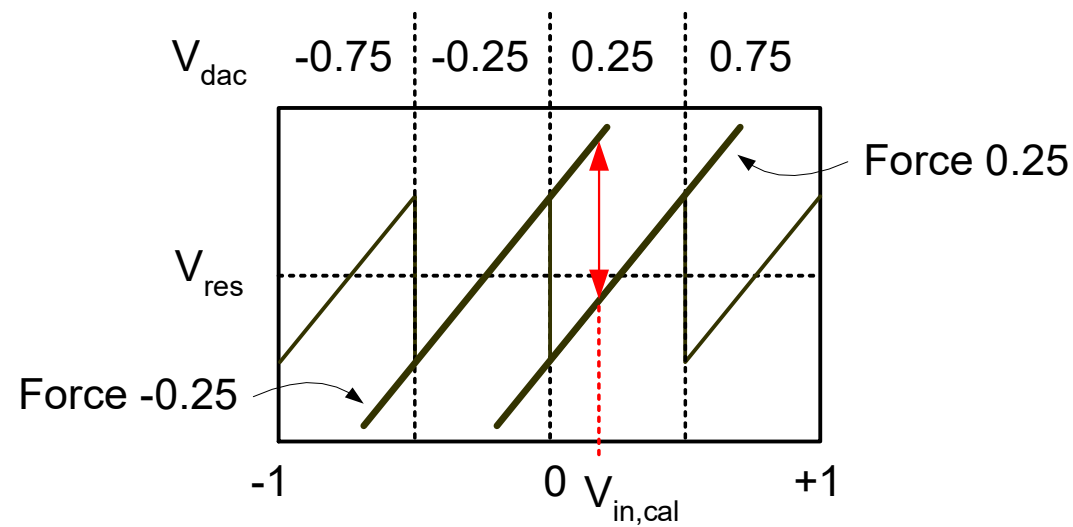
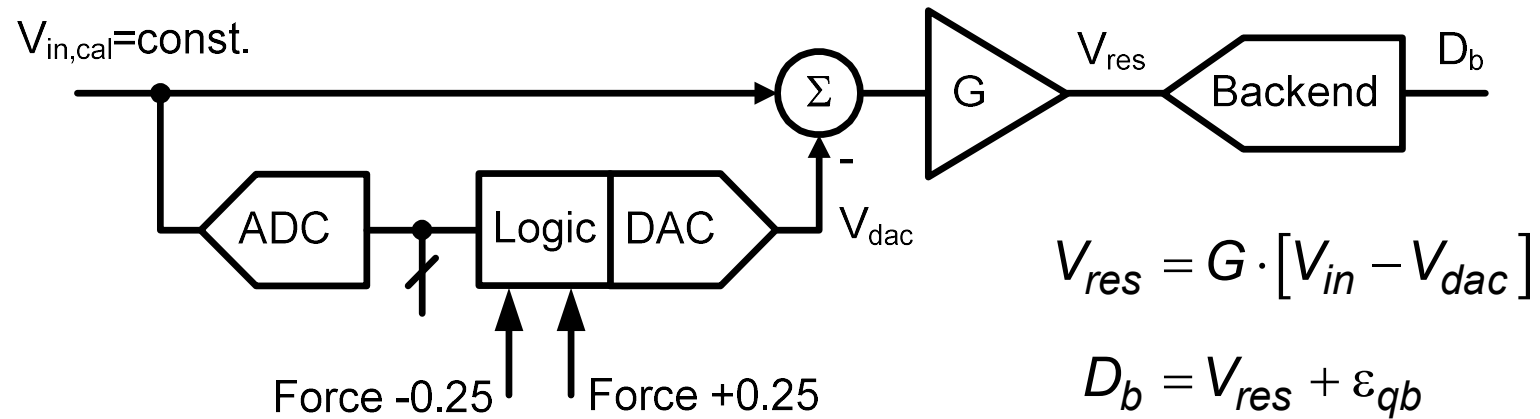
$$D_{out} = V_{in} + \epsilon_{q1} \left(1 - \frac{G_1 + \Delta}{G_{d1}} \right) + \dots + \frac{\epsilon_{qn}}{\prod_{j=1}^{n-1} G_{dj}}$$

- Want to make $G_{d1} = G_1 + \Delta$

Digital Gain Calibration (1)

- Key idea
 - Analog gain error is not a problem as long as **digital gain matches analog gain**
 - Need to measure analog gain precisely
 - Use the backend

Digital Gain Calibration (2)



Digital Gain Calibration (3)

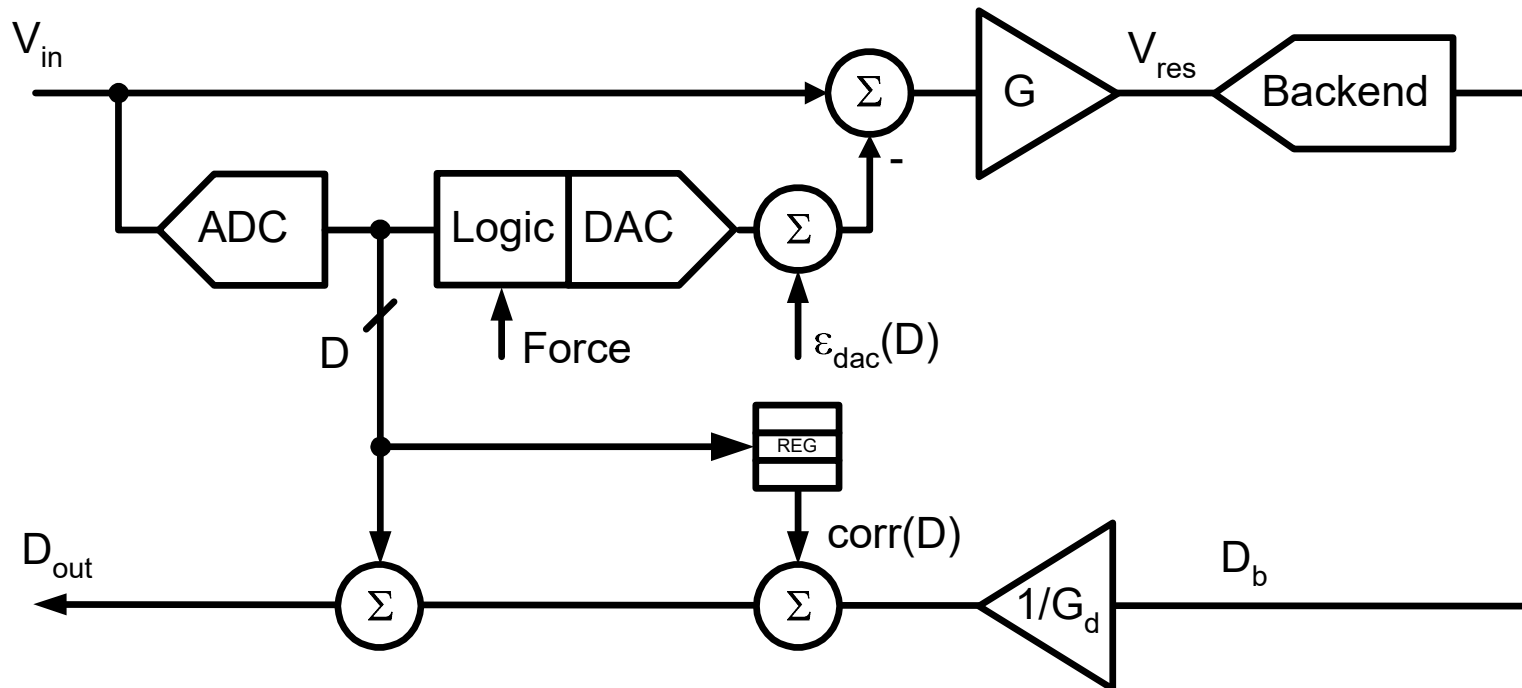
$$\text{Step1: } D_b^{(1)} = G \cdot [V_{in} + 0.25] + \varepsilon_{qb}^{(1)}$$

$$\text{Step2: } D_b^{(2)} = G \cdot [V_{in} - 0.25] + \varepsilon_{qb}^{(2)}$$

$$D_b^{(1)} - D_b^{(2)} = 0.5 \cdot G + \varepsilon_{qb}^{(1)} - \varepsilon_{qb}^{(2)}$$

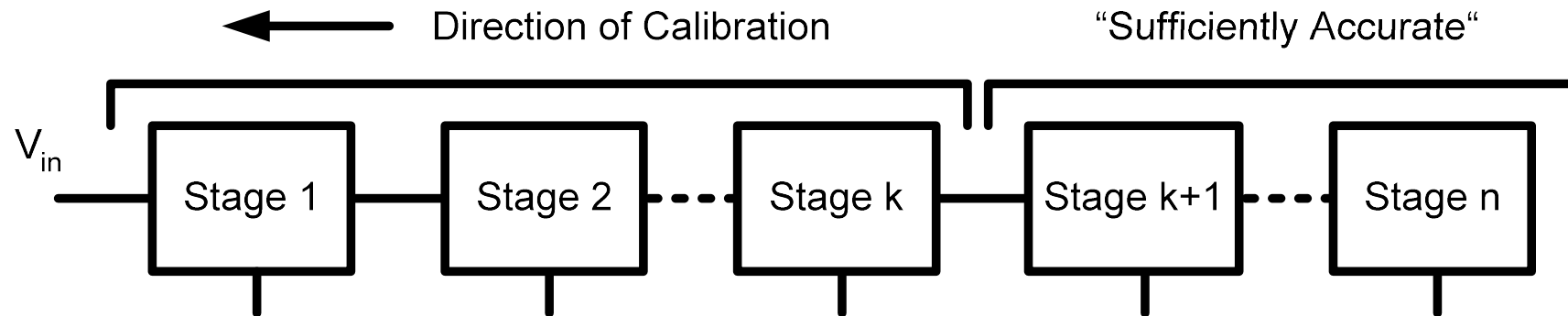
- We can minimize impact of quantization error by
 - Averaging (thermal noise dither)
 - Adding extra backend resolution
- Gain nonlinearity limits ENOB to about 10 bits.

DAC Calibration



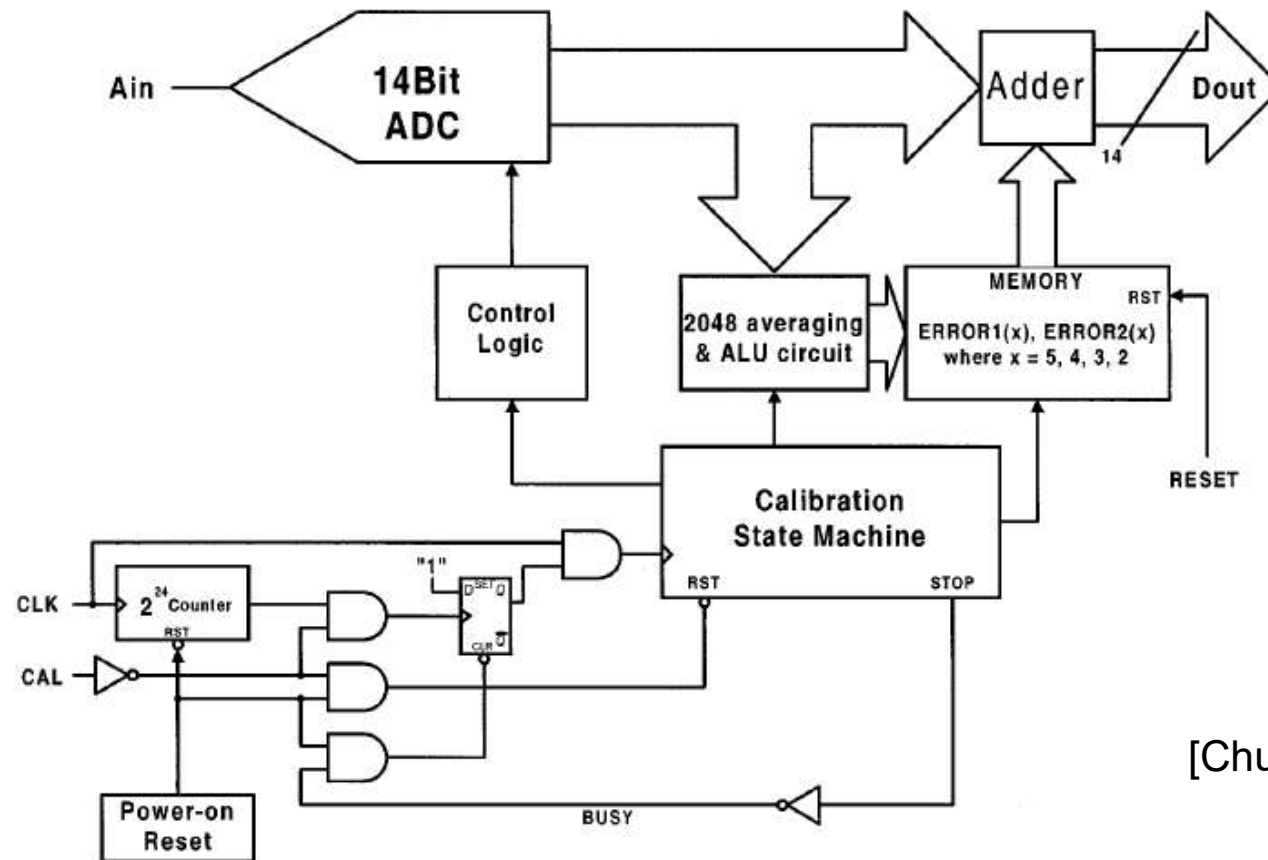
- Same concept as gain calibration
 - Sweep DAC codes and use backend to measure errors
- Store coefficients for each DAC transition in a look-up table
- What if the backend is not accurate?

Recursive Stage Calibration



- First few stages have most stringent accuracy requirements
 - Errors of later stages are attenuated by aggregate gain
- Commonly used algorithm [Karanicolas 1993]
 - Take ADC offline
 - First calibrate the least significant stage that needs calibration
 - Move to the next significant stage and continue toward stage 1

Calibration Hardware Example



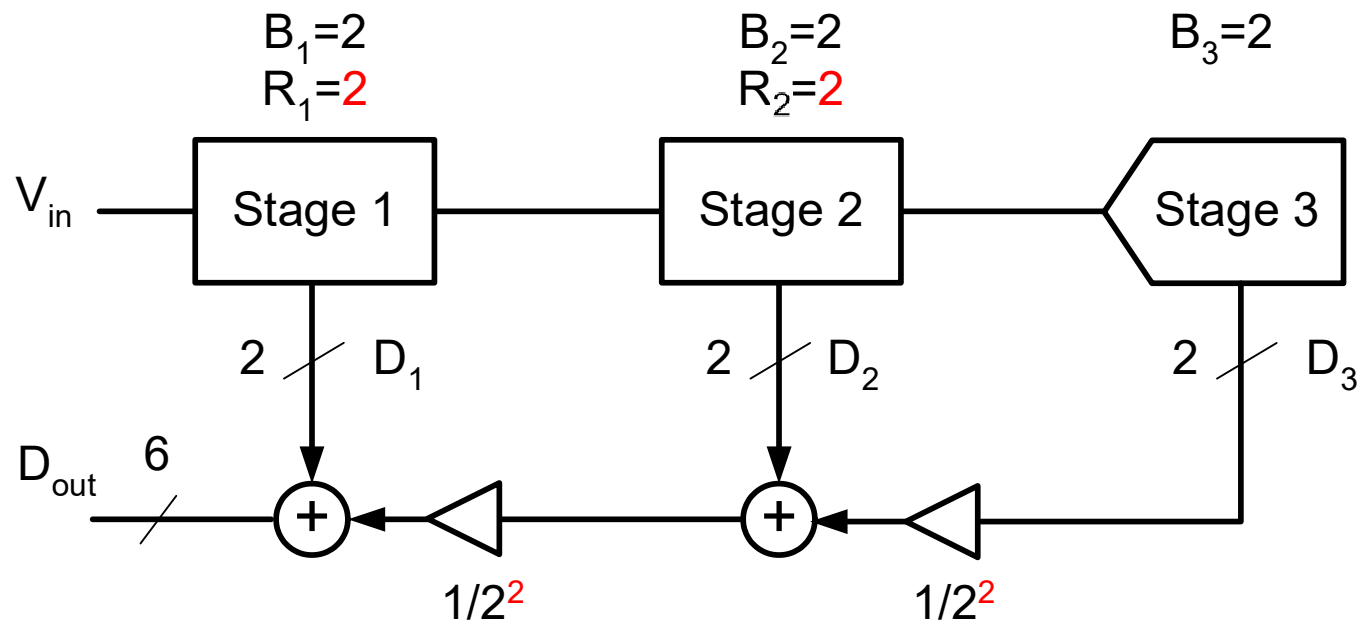
[Chuang 2002]

Alternative Schemes

- Other foreground calibration schemes
 - Calibrate ADC starting from first stage [Singer 2000]
 - Connect stages in a circular loop [Soenen 1995]
- Foreground calibration is not accurate enough when errors drift
 - Gain error drifts due to temperature variation and device aging
 - DAC error due to capacitor mismatch does not drift
- Background calibration techniques [e.g., Ming 2001]
 - Can keep track of gain variations 😊
 - Without disturbing the ADC normal operation 😊

Combining the Bits (1)

- Example1: Three 2-bit stages, no redundancy



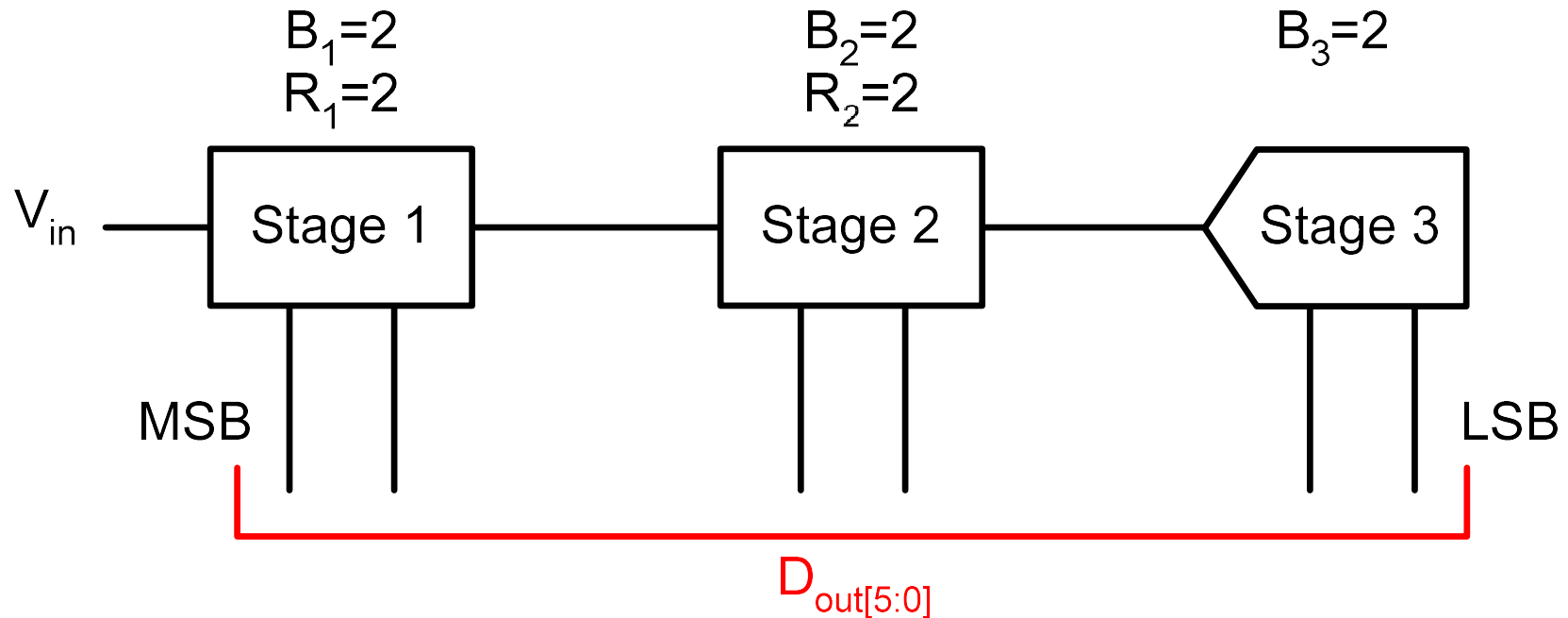
$$D_{out} = D_1 + \frac{1}{4}D_2 + \frac{1}{16}D_3$$

Combining the Bits (2)

D_1 **XX**
 D_2 **XX**
 D_3 **XX**

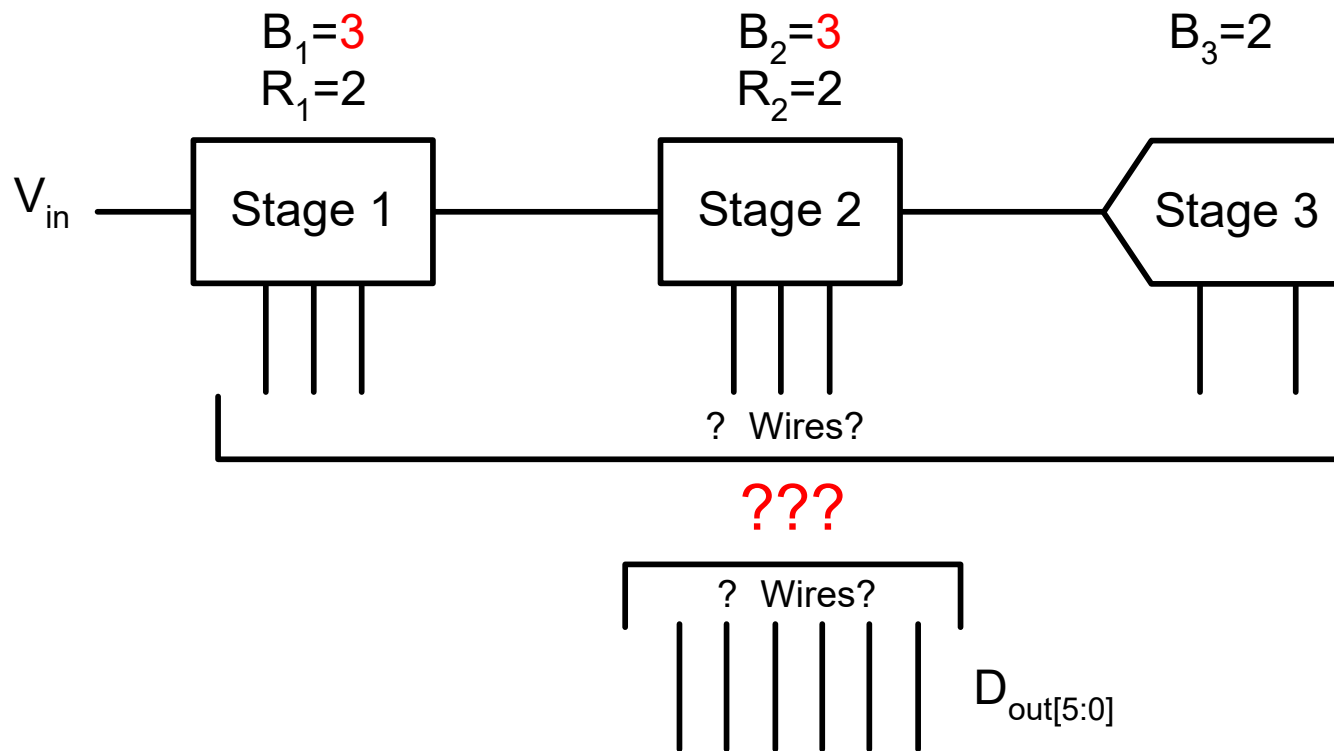
 D_{out} **XXXXXX**

- Only bit alignment
- No digital circuits needed



Combining the Bits (3)

- Example2: 1 bit redundancy in stages 1 and 2

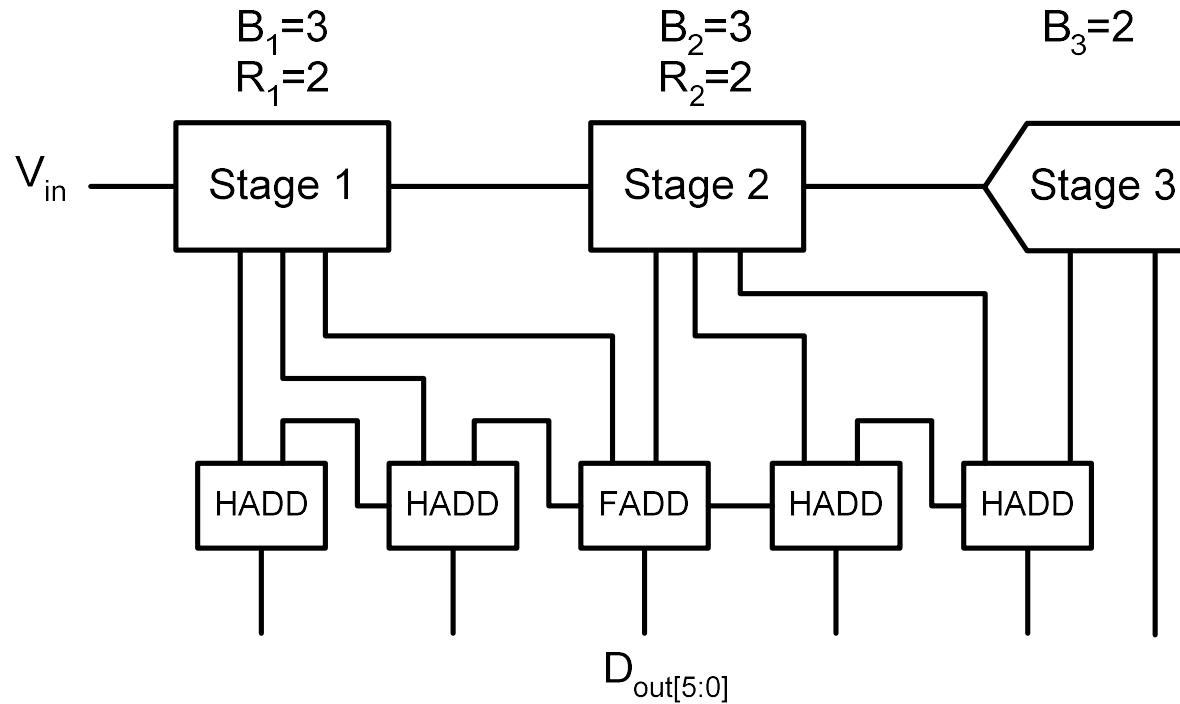


$$D_{out} = D_1 + \frac{1}{4}D_2 + \frac{1}{16}D_3$$

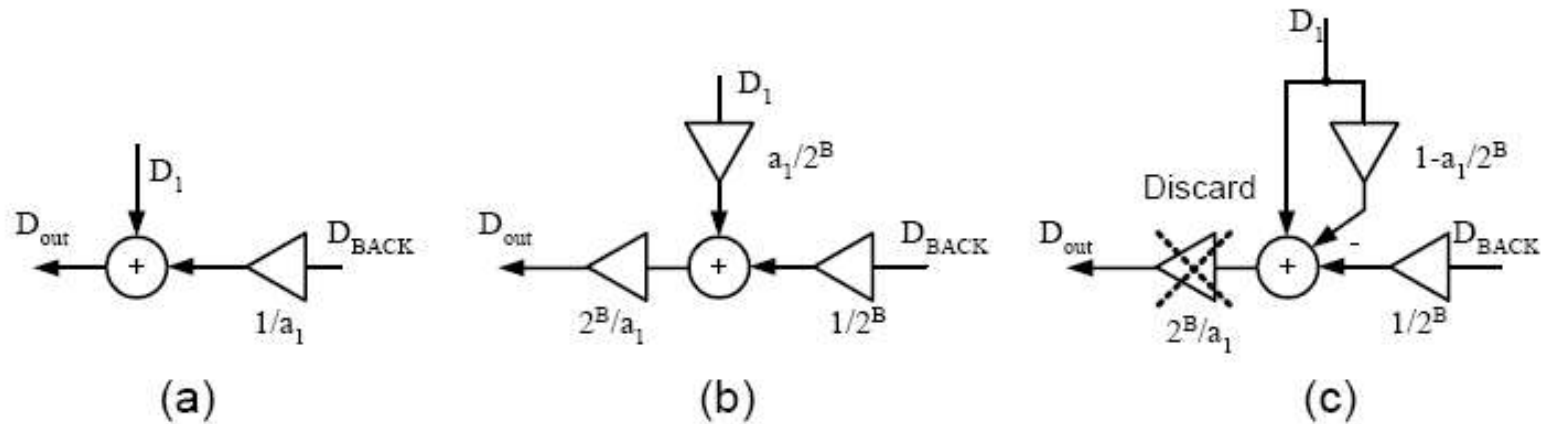
D_1 **XXX**
 D_2 **XXX**
 D_3 **XX**

 D_{out} **DDDDDD**

- Bits overlap
- Need adders



Combining the Bits (5)



- For fractional weights (e.g. radix < 2), avoid the use of complex multipliers
 - $10X = 8X + 2X$
- See e.g. [Karanicolas 1993]